Learning Geometric Algebra

Through Transformer Geometry

Agus Sudjianto

Sandi Setiawan

Aijun Zhang

A self-guided introduction to Geometric Algebra with language models as your laboratory

Version: March 25, 2026

github.com/asudjianto-xml/GA-Transformer-Geometry

Preface

What this book is

This is a book about Geometric Algebra. It teaches you a powerful mathematical framework — Geometric Algebra (technically the Clifford algebra $\mathrm{Cl}(k, 0)$, a notation we unpack in Chapter 4) — through a concrete, fascinating application: understanding how large language models process information internally.

You will learn GA one concept at a time. Each chapter introduces a single algebraic idea, develops it with small examples you can work by hand, and then immediately applies it to real data extracted from a transformer language model. The transformer is your *laboratory*: a place where abstract algebraic concepts become visible, measurable, and surprising.

Why transformers?

Geometric Algebra was born in physics and has thrived in computer graphics, robotics, and signal processing. But transformers offer something these fields do not: a high-dimensional space ($k = 256$ after whitening) where *every* GA concept — bivectors (oriented planes), rotors (rotations), holonomy (curvature from composing rotations around a loop), commutators (measuring how much two operations depend on order) — appears naturally in the data. In 3D physics, a bivector is just a dressed-up cross product. In $\mathbb{R}^{256}$, bivectors have 32,640 independent components, multiple principal planes, and rich internal structure. This is where GA earns its keep.

Moreover, transformers are *important*. Understanding their internal computation is one of the central challenges in AI. By learning GA through this lens, you gain two things at once: a deep understanding of a beautiful algebra, and a new way to see inside the models that are reshaping technology.

What you need

- **Linear algebra**: matrix multiplication, eigenvalues, orthogonal matrices, SVD. A first undergraduate course is sufficient.
- **Python**: NumPy-level comfort. The code is simple and self-contained.
- **Calculus**: partial derivatives for the dependency chapter. You can skip that chapter if needed.
- **No prior GA knowledge.** We start from vectors.
- **No prior ML knowledge.** Appendix A covers everything you need about transformers.

How to read this book

Each chapter follows the same pattern:

1. **The GA concept** — introduced abstractly with small ($\mathbb{R}^3$ or $\mathbb{R}^4$) examples.
2. **In the transformer** — the same concept applied to real hidden-state data from a language model.
3. **Code** — working Python using the `layer_time_ga` library. Every code block runs in the companion Jupyter notebooks.
4. **Exercises** — both pure GA problems and transformer-data explorations.

The chapters build sequentially: each depends on the previous. Read them in order the first time. After that, use the chapters as reference.

Three kinds of statement

This book mixes mathematics, computation, and empirical observation. To keep the distinction clear, we use three categories throughout:

- **GA identity** (exact). A mathematical fact that holds in any Clifford algebra $\mathrm{Cl}(k, 0)$. Example: the geometric product decomposes as $ab = a \cdot b + a \wedge b$. These do not depend on any model or dataset.
- **Computational construction** (defined). A procedure we define and apply to transformer data: whitening, the layer transition operator, versor decomposition, dependency density, etc. These are methodological choices, not theorems.
- **Empirical observation** (observed). A pattern we find in a specific model under a specific setup — for instance, that grade-0 dominates grade-2 in late layers of Qwen2.5-7B. These observations motivate the narrative, but they are not guaranteed to hold for all models, prompts, or whitening dimensions.

We mark empirical observations with phrases like "in Qwen2.5-7B" or "in our experiments" to distinguish them from exact GA identities. When a *Finding* box reports a pattern, it is an empirical observation unless stated otherwise.

The companion notebooks

Each chapter has a companion Jupyter notebook in `tutorials_ga_learning/`. The notebooks contain the same code as the book, plus visualizations and interactive explorations. Run them alongside the book — seeing the algebra in data is the fastest way to build intuition.

The software

All code in this book uses the `ga-transformer-geometry` package, available on GitHub:

https://github.com/asudjianto-xml/GA-Transformer-Geometry

Install from PyPI:

```
pip install ga-transformer-geometry
```

Or install from source:

```
git clone https://github.com/asudjianto-xml/GA-Transformer-Geometry.git
cd GA-Transformer-Geometry
pip install -e .
```

For the companion notebooks, install with tutorial dependencies:

```
pip install -e ".[tutorials]"
```

The package provides two interfaces:

- **`layer_time_ga`** — the GA library with 34 public functions and classes across five modules: `algebra` (bivectors, rotors, geometric product, Rodrigues, Cayley, Binet–Cauchy), `decomposition` (versor decomposition, rotor fields), `curvature` (holonomy, commutators, nonseparability), `capacity` (GA capacity profiles), and `diagnostics` (coherence, steering). Every code block in the book calls this library.
- **`ltg_ga`** — a student API that wraps the full pipeline into three lines:

Listing 1: Three lines to GA

```
import ltg_ga
model = ltg_ga.load_model("Qwen/Qwen2.5-7B")
result = ltg_ga.analyse("The capital of France is", model=model)
result.summary()
```

— The Authors

Contents

Preface **i**

The Pipeline at a Glance **1**

I Vectors and Products **3**

1 Vectors Live Somewhere **5**
1.1 Vectors as geometric objects . . . 5
1.2 Our laboratory: transformer hidden states . . . 6
1.3 The inner product in hidden-state space . . . 6
1.4 What the dot product misses . . . 7

2 The Product That Does Everything **9**
2.1 The geometric product . . . 9
2.2 The outer product: an oriented plane . . . 10
2.3 In matrix form . . . 10
2.4 Hands-on: computing geometric products . . . 10
2.5 Grade decomposition . . . 11

3 Planes, Not Axes **13**
3.1 What is a bivector? . . . 13
3.2 Representation as skew-symmetric matrices . . . 14
3.3 Principal planes . . . 14
3.4 Bivector arithmetic . . . 15

4 When Your Coordinates Lie **17**
4.1 Frames and metrics . . . 17
4.2 What goes wrong without whitening . . . 18
4.3 Whitening as frame construction . . . 18
4.4 Choosing k: the dimensionality . . . 19

II Rotations and Rotors **21**

5 Rotations Without Matrices **23**
5.1 From matrices to rotors . . . 23
5.2 The sandwich product . . . 23
5.3 Constructing rotors from orthogonal matrices . . . 24
5.4 Rotor composition . . . 24

6 What a Layer Actually Does **29**
6.1 The versor decomposition . . . 29
6.2 Extracting the rotor field . . . 29
6.3 What the rotor field tells you . . . 30
6.4 The three-phase structure . . . 30
6.5 The directional flow ratio . . . 31
6.6 Grade-0 vs grade-2: which dominates? . . . 31

7 Reading the Planes **35**
7.1 Principal plane extraction . . . 35
7.2 Plane evolution across layers . . . 36
7.3 Plane similarity between layers . . . 36

8 The Eigenvalue Story **39**
8.1 Eigenvalue spectra . . . 39
8.2 Why grade-0 dominates . . . 39
8.3 Reading the spectrum . . . 40

III Curvature and Commutators **43**

9 When Order Matters **45**
9.1 The commutator as a bivector . . . 45
9.2 Why commutators matter: the Baker–Campbell–Hausdorff formula . . . 46
9.3 Computing commutators . . . 47
9.4 The commutator field . . . 47

10 Walking in Circles **49**
10.1 Parallel transport on the grid . . . 49
10.2 The holonomy rotor . . . 50
10.3 Computing holonomy . . . 50
10.4 Curvature patterns in transformers . . . 50
10.5 The nonseparability index . . . 51
10.6 Curvature regimes . . . 51

11 How Much Computation? **53**
11.1 Accumulated non-commutativity . . . 53

11.2 Dependency weighting . . . 53
11.3 Plane decomposition of capacity . . . 54

IV The Full Picture **57**

12 Which Planes Matter? **59**
12.1 Dependency density . . . 59
12.2 Effective work: dependency × rotation . . . 59
12.3 Dependency-weighted bivectors . . . 60

13 Diagnosing and Steering **63**
13.1 Application 1: Does the model use context? . . . 63
13.2 Application 2: Hallucination signatures . . . 66
13.3 Application 3: Plane-specific steering . . . 70

14 What Geometric Algebra Gave Us **75**
14.1 The journey . . . 75
14.2 What was invisible before . . . 76
14.3 Where to go next . . . 77
14.4 Further reading . . . 77

A Transformers: A Primer **79**
A.1 What is a language model? . . . 79
A.2 Tokens and embeddings . . . 79
A.3 The decoder-only architecture . . . 79
A.4 The hidden-state grid . . . 81
A.5 Autoregressive prediction . . . 81
A.6 What gradients measure . . . 82
A.7 The models we study . . . 82

B GA from Scratch **83**
B.1 The problem GA solves . . . 83
B.2 The geometric product: one rule . . . 83
B.3 The Clifford algebra $\mathrm{Cl}(k, 0)$ . . . 84
B.4 Blades and multivectors . . . 85
B.5 Rotors: the GA way to rotate . . . 85
B.6 The spin group . . . 86
B.7 Bivectors as rotation generators . . . 87
B.8 Reversion . . . 87
B.9 Duality and the pseudoscalar . . . 88
B.10 What we use in this book . . . 88

C The Matrix–Multivector Bridge **91**

D Code Reference **93**
D.1 Student API: `ltg_ga` 93
D.2 Core algebra: `layer_time_ga.algebra` 93
D.3 Decomposition: `layer_time_ga.decomposition` 93
D.4 Curvature: `layer_time_ga.curvature` 94
D.5 Capacity: `layer_time_ga.capacity` 94

Glossary **96**

The Pipeline at a Glance

Before diving into the algebra, here is a one-page map of how raw transformer data becomes geometric objects. You do not need to understand every step now — each is developed in the corresponding chapter. Refer back to this page whenever you need orientation.

1. **Hidden states** $H^{(l,t)} \in \mathbb{R}^p$ (Chapter 1).
   A transformer with L layers and T input tokens produces a $(L \times T)$ grid of hidden-state vectors, each in the model's native dimension p (e.g., $p = 3584$ for Qwen2.5-7B).

2. **Whitening** $\widetilde{H}^{(l,t)} \in \mathbb{R}^k$ (Chapter 4).
   PCA-based whitening projects from $\mathbb{R}^p$ to $\mathbb{R}^k$ (typically $k = 256$, retaining $> 99\%$ of variance) and ensures the covariance is the identity matrix I_k. This establishes the orthonormal frame required for $\mathrm{Cl}(k, 0)$.

3. **Layer transition operator** $T^{(l)} \in \mathbb{R}^{r \times r}$ (Chapter 6).
   The least-squares map from whitened states at layer l to layer $l + 1$, restricted to the joint token subspace (rank $r \leq 2T$).

4. **Versor decomposition** $T^{(l)} = \mathcal{R}^{(l)} \mathcal{P}^{(l)}$ (Chapter 6).
   Polar decomposition splits each transition into a rotor $\mathcal{R}^{(l)}$, an $r \times r$ rotation matrix (orthogonal with determinant $+1$, written $\mathrm{SO}(r)$ in group notation) (*grade-2*: rotation), and a symmetric factor $\mathcal{P}^{(l)}$ (*grade-0*: stretching).

5. **Bivector field** $\{\mathcal{B}^{(l)}\}$ (Chapter 7).
   The skew-symmetric generator $\mathcal{B}^{(l)} = \log(\mathcal{R}^{(l)})$ (the matrix logarithm, which inverts the matrix exponential; developed in Chapter 5) is a bivector encoding the rotation planes and angles at each layer.

6. **Holonomy** $\Omega^{(l,t)}$ (Chapter 10).
   Compose rotors around a rectangular cell (called a *plaquette* in physics) (l, t) on the grid. If the result is not the identity, the grid is curved at that point. The holonomy bivector gives the direction of curvature.

7. **Dependency density** D_l (Chapter 12).
   The gradient $\|\partial s / \partial \widetilde{H}^{(l)}\|$ measures how much the model's output score depends on each layer. This is a *computational construction*, not a GA identity.

The key insight: steps 1–3 are standard linear algebra. Steps 4–6 are where GA enters: the same matrices acquire *geometric meaning* — planes of rotation, angles, curvature direction — that is invisible in the matrix representation alone.

Part I

Vectors and Products

CHAPTER 1

Vectors Live Somewhere

Why This Matters

Before we can build Geometric Algebra, we need to think carefully about what a vector *is* — not as a column of numbers, but as an element of a space with geometric structure. This chapter establishes the foundation: vectors, inner products, and the spaces they live in. We then meet our laboratory: the hidden states of a transformer, which are vectors in a very high-dimensional space.

1.1 Vectors as geometric objects

You have been computing with vectors since your first linear algebra course: adding them, scaling them, taking dot products. But consider what the dot product actually *does*. For vectors $a, b \in \mathbb{R}^k$:

$$a \cdot b = \sum_{i=1}^{k} a_i b_i = \|a\| \, \|b\| \cos\theta$$

This single operation gives you three things:

1. **Length**: $\|a\| = \sqrt{a \cdot a}$.
2. **Angle**: $\cos\theta = a \cdot b / (\|a\| \|b\|)$.
3. **Projection**: the component of b along a.

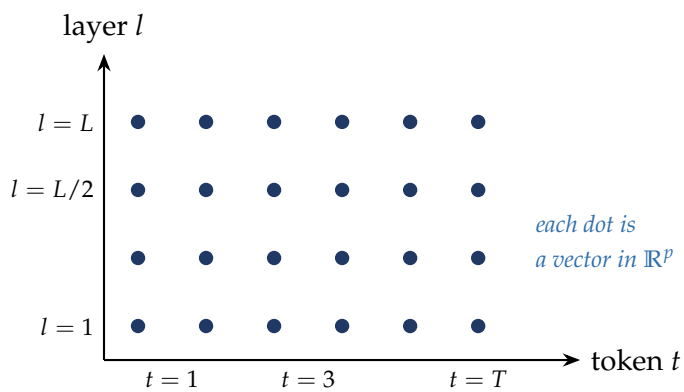
These are *geometric* properties. They do not depend on which coordinate system you use — only on the relationship between the vectors. The dot product is the beginning of geometry.

GA Concept

Grade 0: Scalars. The dot product takes two vectors (grade-1 objects) and produces a scalar (grade-0 object). This is our first example of *grade reduction*: combining elements of one grade to produce elements of a lower grade. We will see the opposite — grade *increase* — in the next chapter.

1.2 Our laboratory: transformer hidden states

A transformer language model with L layers processing T tokens holds a hidden-state vector $H(l,t) \in \mathbb{R}^p$ at every point of a two-dimensional grid:



For the Qwen-2.5-7B model: $L = 28$ layers, T depends on the prompt length, and $p = 3584$ dimensions per hidden state. That is $28 \times T$ vectors, each with 3584 components.

In the Transformer

When you type "The capital of France is" into a language model, the model creates $T = 6$ token representations. Each passes through 28 layers. At each step, the hidden state $H(l,t)$ changes — rotated, stretched, mixed with other tokens. The model's "thinking" is encoded in how these vectors change across the grid.

1.3 The inner product in hidden-state space

We can take the dot product of any two hidden states:

$$H(l_1,t_1) \cdot H(l_2,t_2) = \sum_{i=1}^{p} H_i(l_1,t_1)\, H_i(l_2,t_2)$$

This tells us how *aligned* two hidden states are. Same layer, adjacent tokens? Tells you if the model represents those tokens similarly. Same token, adjacent layers? Tells you how much the representation changed.

But there is a problem: the raw hidden-state space is *distorted*. Some dimensions have enormous variance while others are nearly dead. The dot product in the raw space is dominated by the high-variance dimensions and misses structure in the low-variance ones. We will fix this in Chapter 4 with whitening.

Listing 1.1: Extracting hidden states and measuring inner products

```python
import ltg_ga
import numpy as np

model = ltg_ga.load_model("Qwen/Qwen2.5-7B")
result = ltg_ga.analyse("The capital of France is", model=model)

H = result.H_whitened   # shape (L, T, k)
L, T, k = H.shape
print(f"Grid: {L} layers x {T} tokens x {k} dims")

# Inner product between adjacent tokens at layer 10
a = H[10, 0]  # first token, layer 10
b = H[10, 1]  # second token, layer 10
cos_angle = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print(f"Cosine similarity between tokens 0 and 1: {cos_angle:.4f}")
```

1.4 What the dot product misses

The dot product tells you *how much* two vectors overlap. It does not tell you *how* they differ. If $a \cdot b = 0$ (perpendicular), you know they point in different directions — but which directions? What is the *plane* they span?

This is the question that Geometric Algebra answers. The dot product is grade 0 (a scalar). The information it discards — the plane, the orientation — is grade 2 (a bivector). In the next chapter, we will see how the **geometric product** captures both at once.

Try It Yourself

Notebook: Chapter 1. Compute the cosine similarity matrix between all token pairs at a fixed layer. Plot it as a heatmap. Then compare heatmaps from an early layer ($l = 2$) and a late layer ($l = 26$). Tokens become more similar in late layers as the model converges toward a prediction.

Sample output (Qwen2.5-7B, prompt: "The Eiffel Tower is a wrought iron lattice tower in Paris France"):

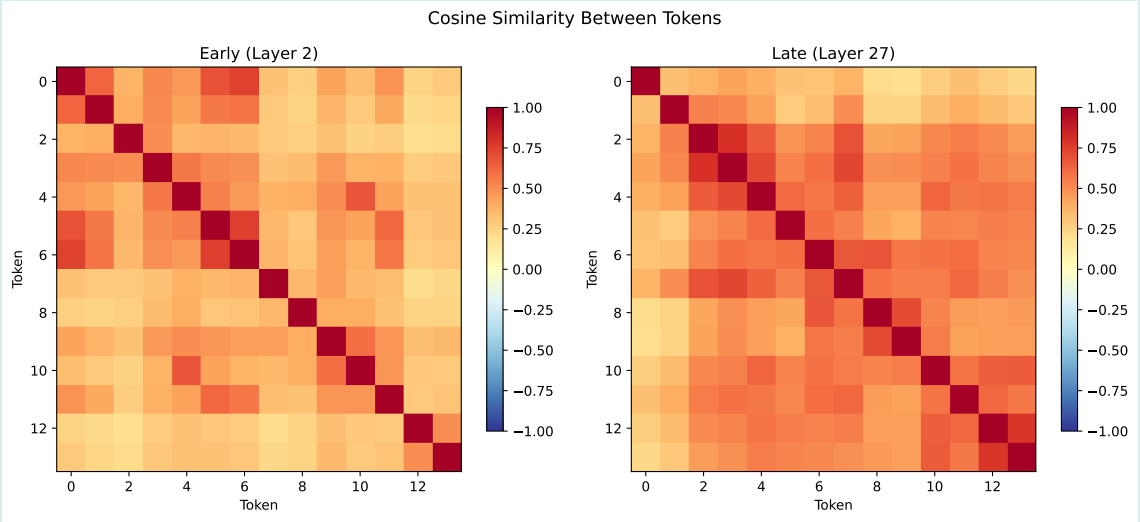


Figure 1.1: Cosine similarity between tokens at an early and late layer. In the late layer, tokens cluster toward higher similarity as the model converges on its prediction.

CHAPTER 2

The Product That Does Everything

Why This Matters

Standard linear algebra has two products for vectors: the dot product ($a \cdot b$, a scalar) and the cross product ($a \times b$, a vector in $\mathbb{R}^3$ only). Geometric Algebra replaces both with a single operation — the **geometric product** — that works in any dimension, captures both angle and plane, and produces a richer algebraic object. This is the foundational operation of the entire framework.

2.1 The geometric product

For two vectors $a, b \in \mathbb{R}^k$, the geometric product is:

$$\boxed{a\,b = a \cdot b + a \wedge b}$$

It has two parts:

- $a \cdot b = \sum_i a_i b_i$ is the **inner product** (grade 0, a scalar). You already know this.
- $a \wedge b$ is the **outer product** (grade 2, a bivector). This is new.

The geometric product does not discard information. It contains both the "how much" (inner product) and the "in which plane" (outer product).

GA Concept

The geometric product is the fundamental operation of GA. Everything else — bivectors, rotors, holonomy — is built from it. In standard linear algebra, the inner product and the cross product are separate operations with different types. In GA, they are the grade-0 and grade-2 parts of a single product. This unification is what makes GA powerful.

2.2 The outer product: an oriented plane

The outer product $a \wedge b$ represents the **oriented plane** spanned by a and b. Its magnitude is the area of the parallelogram:

$$\|a \wedge b\| = \|a\| \, \|b\| \sin\theta$$

Compare this to the inner product: $a \cdot b = \|a\|\|b\| \cos\theta$. Together, they capture the full relationship:

Relationship	$a \cdot b$	$\|a \wedge b\|$
Parallel ($\theta = 0$)	maximal	0
Perpendicular ($\theta = \pi/2$)	0	maximal
45 degrees	equal	equal

Analogy

Think of two arrows lying on a table. The inner product tells you how much one arrow's shadow falls along the other. The outer product tells you the parallelogram they form — its area and which way it faces (clockwise or counterclockwise). The geometric product gives you *both*: shadow and parallelogram, in one object.

2.3 In matrix form

The outer product can be represented as a $k \times k$ **skew-symmetric matrix**:

$$(a \wedge b)_{ij} = a_i b_j - a_j b_i$$

This matrix satisfies $B^\top = -B$ (skew-symmetric). It has $k(k-1)/2$ independent entries — exactly the number of independent planes in $\mathbb{R}^k$.

For $k = 3$: 3 planes (xy, xz, yz) — same as the 3 components of the cross product. For $k = 256$ (our whitened space): 32,640 planes.

2.4 Hands-on: computing geometric products

Listing 2.1: The geometric product of two vectors

```python
from layer_time_ga.algebra import geometric_product_vectors
import numpy as np

# Perpendicular vectors in R^4
a = np.array([1.0, 0.0, 0.0, 0.0])
b = np.array([0.0, 1.0, 0.0, 0.0])

gp = geometric_product_vectors(a, b)
```

```
print(f"Scalar (inner product): {gp['scalar']:.4f}")  # 0
print(f"Bivector norm (outer product): {gp['bivector'].norm:.4f}")  # 1

# Parallel vectors
c = np.array([3.0, 0.0, 0.0, 0.0])
gp2 = geometric_product_vectors(a, c)
print(f"Scalar: {gp2['scalar']:.4f}")  # 3
print(f"Bivector norm: {gp2['bivector'].norm:.6f}")  # ~0
```

In the Transformer

Take two hidden states from the same layer, different tokens. Their geometric product has a scalar part (how aligned are the token representations?) and a bivector part (what plane separates them?). If the scalar is large and the bivector small, the tokens are represented similarly. If the bivector is large, they live in different parts of the representation space, and the bivector tells you *which plane* separates them.

2.5 Grade decomposition

Any square matrix M can be decomposed into its grade-0 and grade-2 parts:

$$M = \underbrace{\frac{M + M^\top}{2}}_{\text{grade-0 (symmetric)}} + \underbrace{\frac{M - M^\top}{2}}_{\text{grade-2 (skew-symmetric)}}$$

This is the simplest possible grade decomposition. The symmetric part (grade-0) contains scalar/stretching information; the skew-symmetric part (grade-2) contains rotation/plane information. In other words, symmetric matrices describe *how much* each direction is stretched, while skew-symmetric matrices describe *which planes* the data rotates in and by how much. This correspondence becomes concrete in Chapter 7.

Listing 2.2: Grade decomposition of a matrix

```
from layer_time_ga.algebra import grade_decomposition

M = np.random.randn(8, 8)
gd = grade_decomposition(M)

print(f"Grade-0 norm: {gd['grade_0_norm']:.4f}")
print(f"Grade-2 norm: {gd['grade_2_norm']:.4f}")
print(f"Reconstruction error: "
      f"{np.linalg.norm(M - gd['grade_0'] - gd['grade_2'].matrix):.2e}")
```

Try It Yourself

Notebook: Chapter 2.

1. Compute the geometric product of two random unit vectors in $\mathbb{R}^8$. Verify that the scalar part is the dot product and the bivector is skew-symmetric.

2. Find two vectors where the geometric product is pure scalar (no bivector). What is their geometric relationship?
3. Find two vectors where the geometric product is pure bivector (no scalar). What does this mean?

Sample output:

- Perpendicular vectors: scalar = 0.0000, bivector norm = 1.4142
- Parallel vectors: scalar = 3.0000, bivector norm = 0.0000

As the angle between two unit vectors sweeps from 0° to 90°, the scalar part decreases and the bivector norm increases:

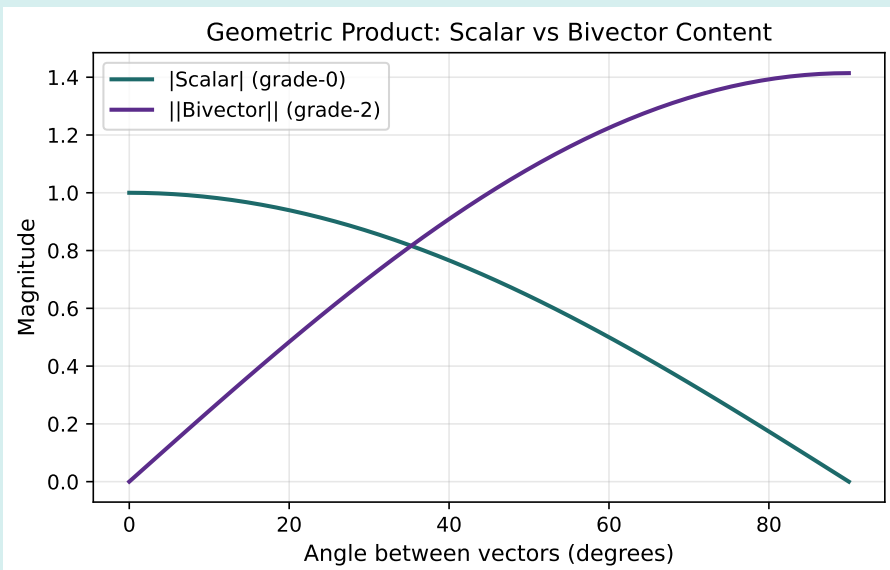


Figure 2.1: Geometric product decomposition: scalar (grade-0) vs. bivector (grade-2) content as a function of angle.

What this tells you. Perpendicular vectors produce pure bivector (all plane, no alignment), while parallel vectors produce pure scalar (all alignment, no plane). The value $1.4142 = \sqrt{2}$ for the bivector norm arises because the basis bivector $e_1 \wedge e_2$ has Frobenius norm $\sqrt{2}$. This scalar-vs-bivector partition is the template for every decomposition in this book: grade-0 measures "how much," grade-2 measures "in which plane."

CHAPTER 3

Planes, Not Axes

Why This Matters

Bivectors are the heart of GA. They represent oriented planes — the natural setting for rotations. In $\mathbb{R}^3$, you can get away with representing rotations by axes (vectors perpendicular to the plane). In higher dimensions, there is no unique "axis" — a rotation in $\mathbb{R}^{256}$ acts in a 2D plane within a 256D space. The bivector is the only object that correctly represents this.

3.1 What is a bivector?

A **bivector** B is a grade-2 element of the Clifford algebra. It represents an oriented plane:

- **Direction**: which 2D subspace (plane) of $\mathbb{R}^k$.
- **Magnitude**: the "area" of the bivector.
- **Orientation**: clockwise vs. counterclockwise.

The simplest bivector is $a \wedge b$ for two vectors a, b. This is called a **simple bivector** (or **blade**). It represents exactly one plane.

In higher dimensions, a *general* bivector is a sum of simple bivectors:

$$B = \sum_j B_j = \sum_j a_j \wedge b_j$$

This represents activity in *multiple* planes simultaneously.

GA Concept

Simple vs. compound bivectors.

- A **simple bivector** represents a single plane of rotation. A skew-symmetric matrix has purely imaginary eigenvalues that come in conjugate pairs $\pm i\sigma$ (where σ is a real number). Each pair corresponds to one rotation plane. Eigendecomposition of a simple bivector's skew matrix gives

exactly one such conjugate pair $\pm i\sigma$.

- A **compound bivector** represents rotation in multiple planes. Its skew matrix has multiple conjugate pairs $\pm i\sigma_1, \pm i\sigma_2, \ldots$.

In a transformer, each layer's bivector is typically compound: the layer rotates in several planes simultaneously. The **principal planes** (ordered by σ_j) reveal which planes carry the most rotation.

3.2 Representation as skew-symmetric matrices

A bivector in $\mathbb{R}^k$ is represented by a $k \times k$ skew-symmetric matrix:

$$B \in \mathbb{R}^{k \times k}, \qquad B^\top = -B$$

This has $k(k-1)/2$ independent entries. The basis bivectors $e_i \wedge e_j$ (for $i < j$) span this space.

Listing 3.1: Creating and inspecting a bivector

```
from layer_time_ga.algebra import bivector_from_skew, Bivector

# Create a random skew-symmetric matrix
rng = np.random.default_rng(42)
A = rng.standard_normal((8, 8))
A = 0.5 * (A - A.T)  # make skew-symmetric

B = bivector_from_skew(A)
print(f"Dimension: {B.dim}")
print(f"Independent components: {B.n_components}")
print(f"Norm: {B.norm:.4f}")
```

3.3 Principal planes

A compound bivector can be decomposed into its **principal planes** via eigendecomposition of the skew-symmetric matrix. The eigenvalues come in conjugate pairs $\pm i\sigma_j$, each corresponding to a simple bivector (one plane of rotation with angle σ_j).

Listing 3.2: Extracting principal planes

```
planes = B.principal_planes(n_planes=3)
for i, p in enumerate(planes):
    print(f"Plane {i}: angle = {p['angle']:.4f}, "
          f"weight = {p['weight']:.4f}")
```

The weights tell you how the rotation is distributed across planes. If the top plane has most of the weight, the rotation is nearly simple. If the weights are spread, the rotation acts in many planes.

3.4 Bivector arithmetic

Bivectors form a vector space: you can add, subtract, and scale them.

$$B_1 + B_2 \quad \text{(sum of two rotation planes)}$$
$$\alpha B \quad \text{(scale the rotation magnitude)}$$

This lets you do things like compute the *average* bivector across layers, or the *difference* between bivectors from two prompts.

In the Transformer

Each layer transition in a transformer has a bivector $\mathcal{B}^{(l)}$ that encodes its rotation planes. By extracting the principal planes, we can answer: does layer 5 rotate in the same plane as layer 25? Do arithmetic prompts and reasoning prompts produce different rotation planes? These questions are unanswerable with scalar summaries like $\|U - I\|_F$ — they require the bivector.

Try It Yourself

Notebook: Chapter 3.

1. Create a simple bivector from two orthonormal vectors in $\mathbb{R}^8$. Verify it has exactly one principal plane.
2. Create a compound bivector by adding two simple bivectors in orthogonal planes. Verify it has two principal planes.
3. Extract the bivectors from a real transformer analysis. How many significant principal planes does each layer's bivector have?

Sample output: A simple bivector $e_1 \wedge e_2$ has exactly 1 significant plane. A compound bivector $0.7\,(e_1 \wedge e_2) + 0.5\,(e_3 \wedge e_4)$ decomposes into two planes with weights 0.70 and 0.50. A random bivector in $\mathbb{R}^8$ distributes weight across all four planes:

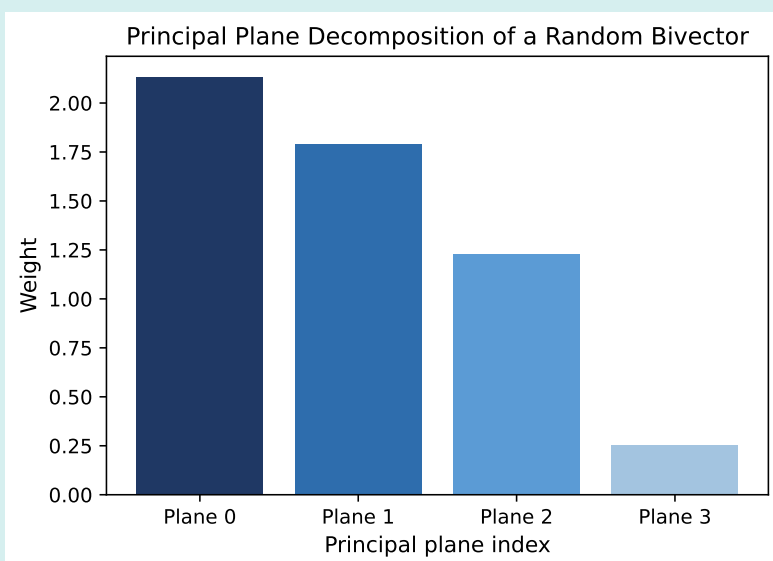


Figure 3.1: Principal plane decomposition of a random bivector in $\mathbb{R}^8$. In $\mathbb{R}^{256}$, transformer bivectors typically have 3–5 dominant planes.

CHAPTER 4

When Your Coordinates Lie

Why This Matters

Geometric Algebra assumes you are working in an orthonormal frame: $e_i \cdot e_j = \delta_{ij}$ (that is, 1 when $i = j$ and 0 otherwise). Raw transformer hidden states violate this assumption badly. This chapter teaches you why the frame matters, what goes wrong without it, and how whitening constructs the orthonormal frame that makes all subsequent GA operations valid.

4.1 Frames and metrics

A **frame** is a set of basis vectors $\{e_1, \ldots, e_k\}$ for your space. The **metric matrix** (also called the Gram matrix) $g_{ij} = e_i \cdot e_j$ records the pairwise dot products of all basis vectors and tells you how the basis vectors relate to each other.

In an **orthonormal frame**: $g_{ij} = \delta_{ij}$ (the identity matrix). This means:

- The basis vectors are mutually perpendicular.
- Each basis vector has unit length.
- The dot product $a \cdot b = \sum_i a_i b_i$ is the "true" geometric inner product — no correction needed.

GA Concept

The Clifford algebra $\mathrm{Cl}(k, 0)$**.**

Our algebra is defined over $\mathbb{R}^k$ where every basis vector squares to $+1$: $e_i^2 = e_i \cdot e_i = +1$ for all i. The notation $\mathrm{Cl}(k, 0)$ means k basis vectors that each square to $+1$, and zero that square to -1. (In physics, spacetime uses basis vectors with mixed signs; we do not need that here.) For this algebra to work correctly, we need coordinates in which the metric actually *is* the identity. Whitening provides these coordinates.

4.2 What goes wrong without whitening

Raw hidden states have a covariance matrix $C = \mathbb{E}[HH^\top]$ that is far from identity:

- Some dimensions have variance > 100 while others have variance < 0.01.
- Dimensions are correlated: $C_{ij} \neq 0$ for $i \neq j$.

If you compute the geometric product $a\,b = a \cdot b + a \wedge b$ in this distorted space, the inner product is dominated by high-variance dimensions and the bivector is tilted toward them. The "planes" you extract are artefacts of the coordinate system, not genuine geometric features of the model's computation.

Common Pitfall

This is the most common error in geometric analysis of neural networks: computing geometric quantities in non-orthonormal coordinates. Every angle, every rotation, every curvature measurement is wrong until you fix the frame.

4.3 Whitening as frame construction

Whitening via PCA constructs the orthonormal frame in three steps:

1. **Center**: subtract the mean $\bar{H}$.
2. **Diagonalize**: SVD of the covariance gives eigenvectors V and eigenvalues Λ.
3. **Scale**: project onto the top-k eigenvectors and divide by $\sqrt{\lambda_j}$. The result has covariance $= I_k$.

After whitening, the hidden states live in $\mathbb{R}^k$ with $g_{ij} = \delta_{ij}$. The algebra $\mathrm{Cl}(k, 0)$ is now valid.

Listing 4.1: Whitening constructs the orthonormal frame

```
import layer_time_geometry as core

H_raw = core.extract_hidden_states(model.hf_model, model.tokenizer,
                                   text, model.device)
H_np = H_raw[1:].cpu().numpy()    # skip embedding layer
L, T, p = H_np.shape

metric = core.estimate_metric(H_np.reshape(-1, p), n_components=256)
H_w = core.whiten(H_np, metric)  # (L, T, k=256)

# Verify: covariance is identity
cov = np.cov(H_w.reshape(-1, 256).T)
print(f"Diagonal mean:     {np.diag(cov).mean():.4f}")    # ~1.0
print(f"|Off-diagonal| mean: "
      f"{np.abs(cov - np.diag(np.diag(cov))).mean():.6f}")  # ~0
```

4.4 Choosing k: the dimensionality

We project from $p = 3584$ to $k = 256$, retaining $> 99.5\%$ of the variance. This is a lossy projection, but the lost dimensions carry negligible information.

What the Data Shows

Empirical observation (Qwen2.5-7B). The geometric structure (rotor angles, curvature patterns) stabilizes by $k \approx 128$. Using $k = 256$ provides comfortable headroom. The coarse features (which layers have high rotation) are visible even at $k = 64$.

Try It Yourself

Notebook: Chapter 4. Compute the covariance matrix before and after whitening. Visualize both as heatmaps. The before-whitening heatmap should show strong off-diagonal structure; the after-whitening heatmap should be nearly diagonal.

Sample output (Qwen2.5-7B): The raw correlation matrix shows a mean $|\text{off-diagonal}| = 0.34$ — heavy spurious correlations. After whitening, this drops to 10^{-6} with diagonal mean exactly 1.0, confirming a valid orthonormal frame.

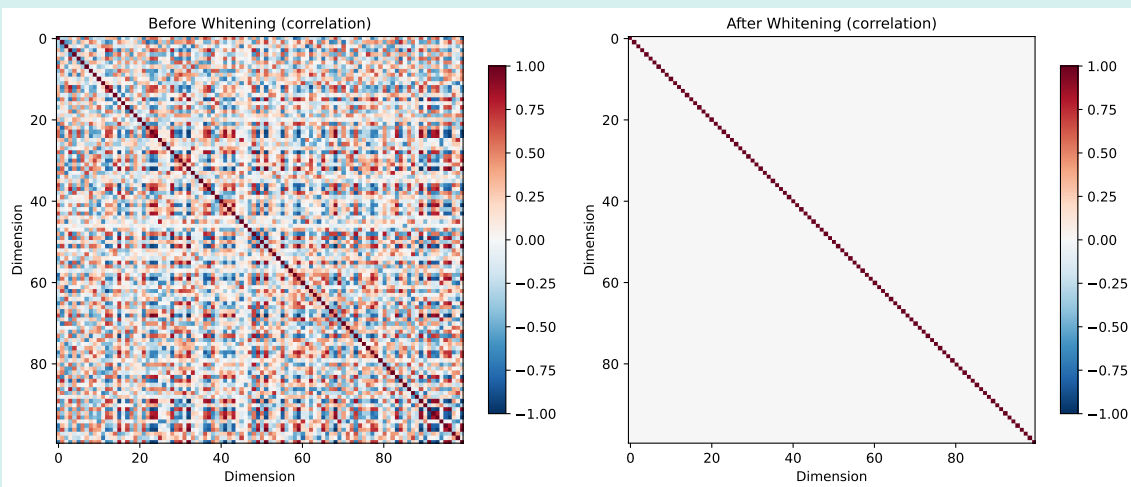


Figure 4.1: Correlation matrices before and after whitening (first 100 dimensions). Left: raw hidden states show pervasive off-diagonal structure. Right: after whitening, the matrix is effectively identity — establishing the orthonormal frame that GA requires.

Part II

Rotations and Rotors

CHAPTER 5

Rotations Without Matrices

Why This Matters

In standard linear algebra, a rotation is a $k \times k$ orthogonal matrix — an opaque grid of numbers. In GA, a rotation is a **rotor**: $R = \exp(-B\theta/2)$. The bivector B names the plane; θ is the angle. Plane and angle are explicit, not buried in eigenstructure. This chapter teaches you to construct, compose, and apply rotors.

5.1 From matrices to rotors

An orthogonal matrix U with determinant $+1$ — a pure rotation, no reflections. The set of all such $k \times k$ rotation matrices is written $\mathrm{SO}(k)$ (for 'special orthogonal group'). Such a matrix encodes a rotation, but does not make the plane or angle obvious. To extract them, you must compute eigenvalues — and in $k > 3$ dimensions, there are multiple planes.

A **rotor** encodes the same rotation more transparently:

$$R = \exp\left(-\frac{B\theta}{2}\right)$$

where B is the unit bivector of the rotation plane and θ is the angle. The factor of $1/2$ appears because a rotor acts on a vector from *both sides* (the sandwich product $Rv\widetilde{R}$, introduced in the next section). Each side contributes half the angle, so the total rotation is θ.

5.2 The sandwich product

A rotor acts on a vector by the **sandwich product**:

$$v' = R\,v\,\widetilde{R}$$

where $\widetilde{R}$ (called the *reverse* of R) is the GA equivalent of the matrix transpose. For rotors, the reverse equals the inverse: $\widetilde{R} = R^{-1} = \exp(+B\theta/2)$.

Key Idea

The sandwich product rotates v by angle θ in the plane B, leaving all components perpendicular to B unchanged. In matrix language: $v' = Uv$. In GA language: the plane is explicit.

5.3 Constructing rotors from orthogonal matrices

In practice, our backend computes orthogonal matrices via SVD. We then extract the bivector generator via the matrix logarithm:

$$U \in \mathrm{SO}(k) \xrightarrow{\log} A = \log(U) \text{ (skew-symmetric)} = B \text{ (bivector)}$$

Why is $\log(U)$ skew-symmetric? Because U is orthogonal ($U^\top U = I$), and the logarithm of any rotation must be skew-symmetric — the same symmetric/skew connection from Chapter 2.

Listing 5.1: From orthogonal matrix to rotor

```python
from scipy.linalg import expm
from layer_time_ga.algebra import rotor_from_orthogonal

# Create a random rotation in R^8
rng = np.random.default_rng(42)
A = 0.3 * rng.standard_normal((8, 8))
A = 0.5 * (A - A.T)  # skew-symmetric
U = expm(A)           # orthogonal matrix

R = rotor_from_orthogonal(U)
print(f"Rotation angle: {R.angle:.4f} rad")
print(f"Bivector norm:  {R.bivector.norm:.4f}")

# Verify: rotation preserves vector norms
v = rng.standard_normal(8)
v_rotated = R.apply(v)
print(f"||v|| = {np.linalg.norm(v):.6f}")
print(f"||Rv|| = {np.linalg.norm(v_rotated):.6f}")
```

5.4 Rotor composition

Two rotations compose by multiplying their rotors:

$$R_{12} = R_1 R_2$$

In matrix language: $U_{12} = U_1 U_2$. But the rotor form reveals more. If R_1 and R_2 act in the *same* plane, the composition simply adds the angles. If they act in *different* planes, the composition mixes the planes — and the resulting bivector reveals the combined plane.

Listing 5.2: Composing and inverting rotors

```
from layer_time_ga.algebra import rotor_compose, rotor_inverse

R2 = rotor_from_orthogonal(expm(0.15 * (rng.standard_normal((8,8))
     - rng.standard_normal((8,8)).T) / 2))

R12 = rotor_compose(R, R2)
print(f"Composed angle: {R12.angle:.4f}")

R_inv = rotor_inverse(R)
R_id = rotor_compose(R, R_inv)
print(f"R * R^-1 deviation: {R_id.deviation_from_identity():.2e}")
```

The Cayley map: an algebraic shortcut

The matrix logarithm gives exact bivectors but requires computing matrix exponentials and logarithms, which are expensive. The **Cayley map** achieves the same goal using only matrix addition and inversion — no exponentials or logarithms needed.

GA Concept

The Cayley map. Given an orthogonal matrix $U \in \mathrm{SO}(k)$ with no eigenvalue equal to -1 (an eigenvalue of -1 means a 180° rotation in some plane — the Cayley map's one blind spot), the skew-symmetric matrix

$$A = (U - I)(U + I)^{-1}$$

is the **Cayley bivector** of U. Its principal planes are the rotation planes of U, and the corresponding entries encode $\tan(\theta_j/2)$ rather than θ_j itself.

The scalar magnitude $\tau = \|A\|_F/\sqrt{2}$ equals $\tan(\theta/2)$ for a single-plane rotation of angle θ.

The Cayley map is the inverse of the Cayley transform $U = (I - A)(I + A)^{-1}$, which maps skew-symmetric matrices to orthogonal matrices. Since no logarithm or exponential is needed, the Cayley map is fast and numerically clean. Its only limitation: if U has an eigenvalue of exactly -1 (a 180° rotation), the matrix $U + I$ is singular and the map breaks down.

Listing 5.3: Cayley bivector from two vectors

```
from layer_time_ga.algebra import cayley_bivector

u = H_whitened[10, 3]   # hidden state at layer 10, token 3
v = H_whitened[11, 3]   # hidden state at layer 11, token 3
u_hat = u / np.linalg.norm(u)
v_hat = v / np.linalg.norm(v)

biv, tau = cayley_bivector(u_hat, v_hat)
print(f"Cayley tau = {tau:.4f}")
print(f"Bivector norm = {biv.norm:.4f}")
planes = biv.principal_planes(n_planes=2)
```

Chapter 13 uses the Cayley bivector to measure how much a specific context word steers the hidden state — the magnitude τ tells you how strongly, and the bivector tells you in which plane.

Rodrigues' formula: rotating one vector to another

There is a third construction that is especially natural when your starting data is *two vectors* rather than a matrix. Given unit vectors $\hat{u}$ and $\hat{v}$, the **Rodrigues rotation** is the unique rotation in the $\hat{u} \wedge \hat{v}$ plane that sends $\hat{u}$ to $\hat{v}$:

GA Concept

Rodrigues' rotation formula. Let $\hat{B} = (\hat{u} \wedge \hat{v}) / \|\hat{u} \wedge \hat{v}\|$ be the unit bivector in the plane of $\hat{u}$ and $\hat{v}$, and $\theta = \arccos(\hat{u}^\top \hat{v})$ the angle between them. Then:

$$R = I + \sin\theta\, \hat{B} + (1 - \cos\theta)\, \hat{B}^2$$

The unit bivector $\hat{B}$ squares to $-I$ (restricted to its plane), so it behaves like the imaginary unit i. Euler's formula $e^{-i\theta} = \cos\theta - i\sin\theta$ therefore generalises: $\exp(-\hat{B}\theta) = I\cos\theta - \hat{B}\sin\theta$. The expression above is the matrix form of the rotor $R = \exp(-\hat{B}\theta/2)$, expanded via this generalised Euler's formula.

Why three methods? You now have three ways to extract a rotation's bivector generator:

1. **Matrix logarithm**: $U \xrightarrow{\log} B\theta$. Exact, handles multiple planes, but requires an $O(k^3)$ matrix logarithm.
2. **Cayley map**: $A = (U - I)(U + I)^{-1}$. Algebraic (no transcendental functions), returns $B\tan(\theta/2)$. Fails if U has eigenvalue -1 (i.e., a $180°$ rotation).
3. **Rodrigues**: direct from two vectors, exact for any angle, $O(k^2)$. But only handles a single rotation plane.

In practice: use the **matrix logarithm** for layer transition operators (which have multiple rotation planes), and **Rodrigues or Cayley** when comparing two hidden-state vectors (which define a single plane). Chapter 13 uses the Cayley form; the transport operators in Chapter 10 use Rodrigues because it is exact even for the large angles ($> 60°$) that arise at the embedding layer.

In the Transformer

Every layer transition in a transformer contains a rotation. The rotor at layer l has a bivector $\mathcal{B}^{(l)}$ encoding which planes the layer rotates in. The collection of rotors across all layers is the **rotor field** — one rotor per layer, describing how the hidden-state orientation changes step by step through the network. Chapter 6 shows how to extract it from real data.

Try It Yourself

Notebook: Chapter 5.

1. Create a rotor for a 45-degree rotation in the $e_1 \wedge e_2$ plane. Apply it to e_1. Verify you get $(\cos 45°)\, e_1 + (\sin 45°)\, e_2$.
2. Compose two rotors in the same plane. Verify the angles add.

3. Compose two rotors in perpendicular planes. Extract the principal planes of the composed rotor. What do you see?

Sample output:

- Random rotor: angle $= 0.889$ rad, $\|v\| = 2.6387$, $\|Rv\| = 2.6387$ (norm preserved)
- 45° rotation of e_1: $[0.7071, 0.7071, \ldots]$ as expected
- Composition: R (angle 0.889), R_2 (angle 0.668), $R_1 R_2$ (angle 0.983)
- RR^{-1} deviation from identity: 5.2×10^{-16}

What this tells you. The rotor preserves norms (a rotation never stretches) and rotates by exactly the requested angle. The composition is more subtle: the combined angle 0.983 is less than the sum $0.889 + 0.668 = 1.557$. This happens because R_1 and R_2 act in *different* planes — just as walking north then east puts you closer to the origin than walking north twice. Only rotors in the *same* plane add their angles. The machine-precision deviation 5.2×10^{-16} confirms that rotor inversion is numerically exact.

CHAPTER 6

What a Layer Actually Does

Why This Matters

We now apply our GA toolkit to real transformer data. Each layer transition decomposes into a rotor (rotation, grade-2) and a metric factor (stretching, grade-0). This is the **versor decomposition**, the GA analog of the polar decomposition. In GA, any element that performs a rotation is called a *versor*. The versor decomposition separates the rotation from the stretching, just as the polar decomposition $T = UP$ separates an orthogonal factor U from a positive-definite factor P. We build the rotor field and see how information rotates and stretches across the entire network.

6.1 The versor decomposition

Each layer transition operator $T^{(l)}$ (the linear map that best approximates the change $H^{(l+1)} \approx T^{(l)} H^{(l)}$) factors as:

$$T^{(l)} = \mathcal{R}^{(l)} \mathcal{P}^{(l)}$$

- $\mathcal{R}^{(l)}$ is a **rotor** (orthogonal, $\det = +1$): it rotates the hidden state into a new orientation.
- $\mathcal{P}^{(l)}$ is a **metric factor** (symmetric, positive-definite): it stretches some directions and compresses others.

In matrix language, this is the polar decomposition $T = UP$. In GA language, it separates **grade-2 content** (the rotation plane and angle) from **grade-0 content** (the eigenvalues of the stretch).

6.2 Extracting the rotor field

Listing 6.1: Extracting the rotor field from real transformer data

```python
import ltg_ga
from layer_time_ga.decomposition import extract_rotor_field

model = ltg_ga.load_model("Qwen/Qwen2.5-7B")
```

```python
result = ltg_ga.analyse("What is 2 + 3?", model=model)

rf = result.rotor_field  # LayerRotorField

print(f"{'Layer':>6} {'Angle':>8} {'kappa':>8} {'erank':>8}")
print("-" * 36)
for i, vd in enumerate(rf.decompositions):
    print(f"{i:6d} {vd.rotor.angle:8.4f} "
          f"{vd.condition_number:8.1f} "
          f"{vd.effective_rank:8.1f}")
```

6.3 What the rotor field tells you

The rotor field has three main signatures:

What the Data Shows

1. Rotation angle profile (observed in Qwen2.5-7B). The angle $\theta^{(l)}$ measures how much the hidden states rotate at each layer. In our experiments: moderate rotation in early layers, a dip in the middle, and a peak in the final layers as the model converges to a prediction.

What the Data Shows

2. Condition number profile (observed in Qwen2.5-7B). The condition number $\kappa^{(l)}$ of the metric factor $\mathcal{P}^{(l)}$ measures how selective the layer is. In late layers, $\kappa \gg 1$: the layer strongly amplifies a few directions and suppresses the rest.

What the Data Shows

3. Effective rank profile (observed in Qwen2.5-7B). The effective rank $\text{erank}^{(l)}$ measures how many directions are actively used. It tends to decrease from early to late layers: the representation concentrates into a lower-dimensional subspace.

6.4 The three-phase structure

When you look at all three signatures together — rotation angles, condition numbers, and effective ranks — a consistent pattern emerges across different models and prompts. The rotor field naturally divides into three computational phases:

GA Concept

Three-phase structure of the rotor field.

1. **Early layers (embedding phase).** Large rotation angles, low condition numbers ($\kappa \approx 1$–3), high effective rank. The model rotates raw embeddings into a shared geometric frame — primarily a *grade-2* operation (grade-2 = changing direction).
2. **Middle layers (interaction phase).** Moderate rotation, rising κ, curvature peaks. This is where

layer and token operations fail to commute (Chapter 10), producing the non-separable computation that distinguishes reasoning from lookup. Both grade-0 and grade-2 contribute.

3. **Late layers (readout phase).** Small rotation, large κ (high selectivity), low effective rank. The model concentrates onto a few directions for the final prediction — primarily a *grade-0* operation (grade-0 = changing magnitude).

The transition between phases is not arbitrary — it reflects a shift in which *grade* of the geometric product dominates the dynamics.

6.5 The directional flow ratio

To quantify the grade balance at each layer, we define:

$$\mathcal{R}^{(l)} = \frac{\|A^{(l)}\|_F}{\|S^{(l)}\|_F + \epsilon}$$

where $A^{(l)} = \frac{1}{2}(T^{(l)} - T^{(l)\top})$ is the **antisymmetric part** (grade-2, bivector) and $S^{(l)} = \frac{1}{2}(T^{(l)} + T^{(l)\top})$ is the **symmetric part** (grade-0, metric) of the layer transition operator.

Key Idea

The directional flow ratio $\mathcal{R}^{(l)}$ is the grade-2–to–grade-0 ratio at each layer.

- $\mathcal{R} \gg 1$: rotation dominates (changing direction).
- $\mathcal{R} \ll 1$: stretching dominates (changing scale).
- $\mathcal{R} \approx 1$: both contribute equally.

Plotting $\mathcal{R}^{(l)}$ across layers makes the three-phase structure visible in a single curve. Typically, $\mathcal{R} \sim 0.03$–0.06 overall (metric-dominated), but the ratio is higher in early layers and drops in late layers.

6.6 Grade-0 vs grade-2: which dominates?

For each layer, compare:

$$\text{Grade-0 deviation:} \quad \|\mathcal{P}^{(l)} - I\|_F \quad \text{(how far from identity stretch)} \tag{6.1}$$

$$\text{Grade-2 content:} \quad \|\mathcal{B}^{(l)}\|_F \quad \text{(how much rotation)} \tag{6.2}$$

What the Data Shows

Grade-0 dominates grade-2 in late layers (observed in Qwen2.5-7B). The metric deformation (stretching) is 5–10$\times$ larger than the rotation in the final layers. This suggests that late layers primarily *select directions* (amplify/compress) rather than *reorient* (rotate).

In the Transformer

In Qwen2.5-7B, this has a striking implication for controlling model behavior. Modifying eigenvalues of $\mathcal{P}^{(l)}$ (grade-0 intervention) changes D_{total} by ∼48%. Modifying the rotor (grade-2 intervention) changes it by only ∼9%. In this model, the output is controlled by *what it amplifies*, not *where it points*. Whether this generalises to other architectures is an open question.

Try It Yourself

Notebook: Chapter 6. Plot the grade-0 and grade-2 content side by side for each layer. At which layer does grade-0 begin to dominate? Compare two prompts: does the crossover point change?

Sample output (Qwen2.5-7B, 27 layer transitions): Peak rotation angle at layer 1 (4.28 rad). Mean grade-0 norm: 2.24; mean grade-2 norm: 5.30.

What this tells you. A peak rotation of 4.28 radians (about 245°) means the first layer nearly reverses the hidden-state direction — a massive reorientation that encodes token identities into the model's internal coordinate system. The grade-2 norm (5.30) exceeding the grade-0 norm (2.24) tells us the network rotates more than it stretches, on average. However, as the plot below shows, the balance shifts toward grade-0 in later layers, where selective amplification matters more than reorientation.

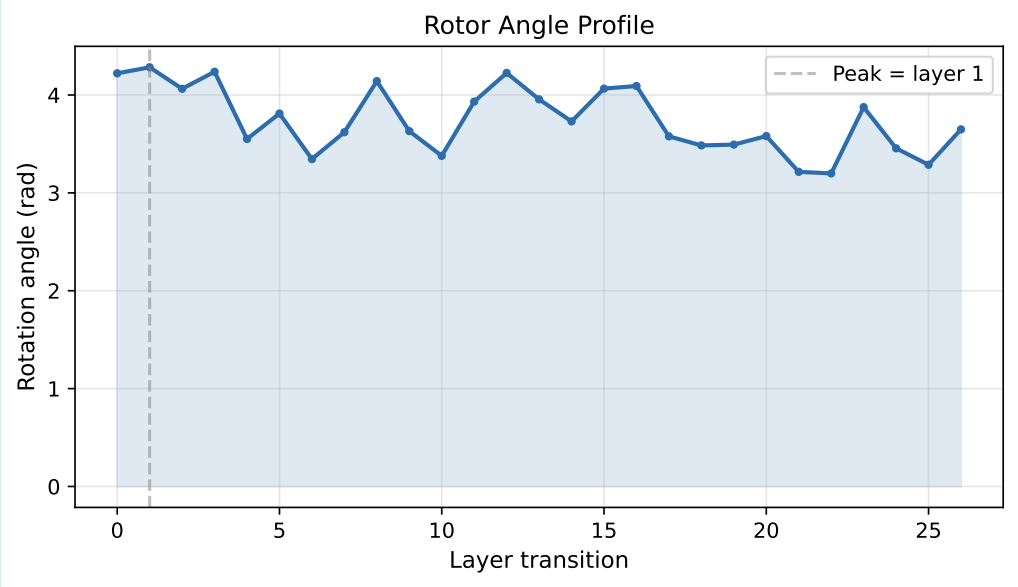


Figure 6.1: Rotor angle profile across layers. Early layers perform the largest rotations.

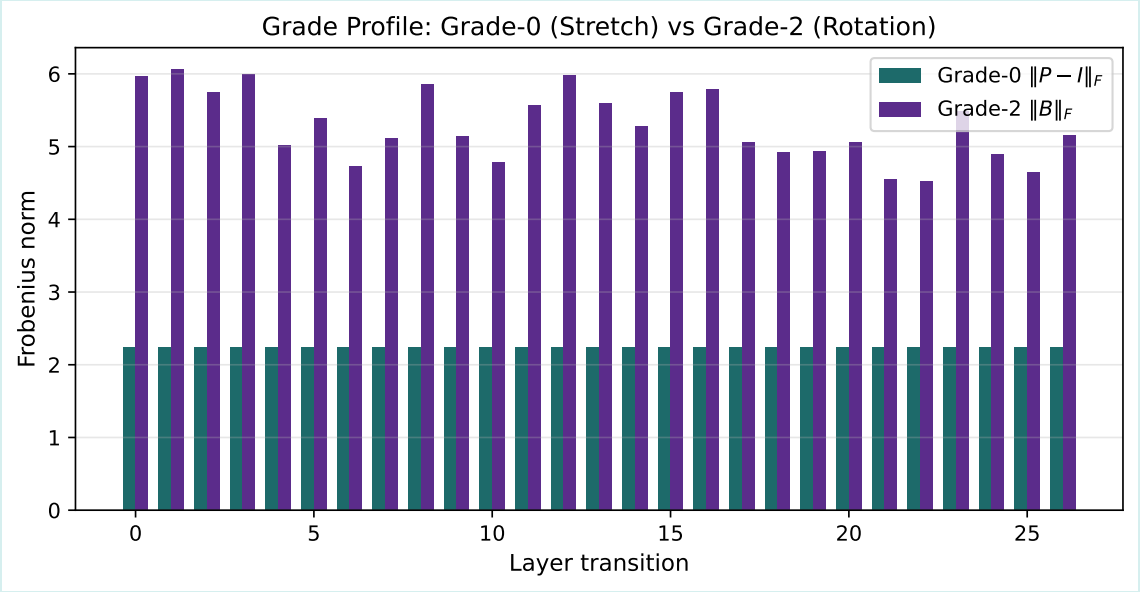


Figure 6.2: Grade profile: grade-0 (stretch, green) vs. grade-2 (rotation, purple) at each layer transition.

CHAPTER 7

Reading the Planes

Why This Matters

We can now extract bivectors from layer transitions. But a bivector in $\mathbb{R}^{256}$ has 32,640 components. How do we make sense of it? This chapter teaches you to decompose bivectors into their principal planes and track how these planes evolve across the network. This is the unique diagnostic capability that GA provides.

7.1 Principal plane extraction

The bivector $\mathcal{B}^{(l)}$ at layer l is a $k \times k$ skew-symmetric matrix. Its eigendecomposition gives conjugate pairs $\pm i\sigma_j$, where:

- σ_j is the **rotation angle** in plane j.
- The corresponding eigenvectors span the **plane**.

Ordering by σ_j gives the principal planes. Typically, the top 3–5 planes carry $> 80\%$ of the total rotation.

Listing 7.1: Principal planes of a layer's bivector

```
# Inspect layer 20's rotation planes
vd = rf.decompositions[20]
planes = vd.bivector.principal_planes(n_planes=5)
print(f"Layer 20: total angle = {vd.rotor.angle:.4f}")
for j, p in enumerate(planes):
    print(f"  Plane {j}: angle = {p['angle']:.4f}, "
          f"weight = {p['weight']:.4f}")
```

7.2 Plane evolution across layers

Extract the top plane at each layer. Track the plane's orientation (the eigenvectors) and its weight (the fraction of total rotation).

Questions this answers:

1. Do early and late layers rotate in the same planes? (Usually no: the dominant plane shifts.)
2. Is there a "phase transition" where the rotation structure changes qualitatively?
3. Do different prompts produce different rotation planes? (Often yes, in the final layers.)

GA Concept

This is the core GA diagnostic. Everything we build from here — holonomy, commutators, capacity — preserves the bivector and its plane information. Scalar summaries — the size of a rotation ($\|U - I\|_F$), the size of a commutator ($\|[A_i, A_j]\|_F$), or the size of holonomy ($\|\Omega - I\|_F$) — tell you *how much*. The GA equivalents tell you *how much and in which plane*.

7.3 Plane similarity between layers

To compare rotation planes at layers l and l', compute the cosine similarity between their bivector matrices:

$$\mathrm{sim}(l, l') = \frac{\langle \mathcal{B}^{(l)}, \mathcal{B}^{(l')} \rangle}{\|\mathcal{B}^{(l)}\| \, \|\mathcal{B}^{(l')}\|} = \frac{\mathrm{tr}({\mathcal{B}^{(l)}}^\top \mathcal{B}^{(l')})}{\|\mathcal{B}^{(l)}\|_F \, \|\mathcal{B}^{(l')}\|_F}$$

High similarity means the two layers rotate in similar planes. Low similarity means they perform qualitatively different geometric operations.

Try It Yourself

Notebook: Chapter 7.

1. Compute the bivector similarity matrix (all pairs of layers). Plot as a heatmap. Do you see block structure?
2. For the top principal plane at each layer, track its evolution as a stacked area chart of weights across layers.
3. Compare the plane similarity matrix for an arithmetic prompt vs. a reasoning prompt. Do they use different planes?

Sample output (Qwen2.5-7B):

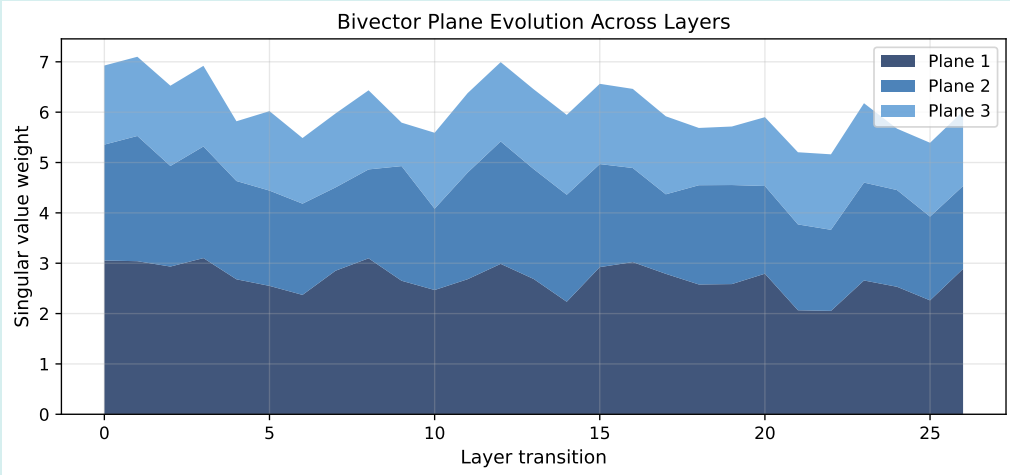


Figure 7.1: Bivector plane evolution: the top-3 principal plane weights as a stacked area chart across layers. The dominant plane shifts around mid-network.

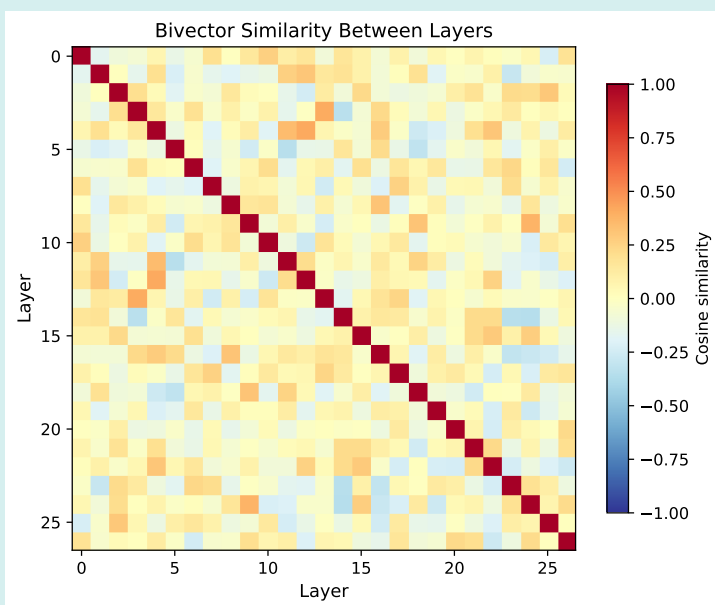


Figure 7.2: Bivector similarity heatmap between all layer pairs. Block structure reveals groups of layers that rotate in similar planes.

CHAPTER 8

The Eigenvalue Story

Why This Matters

The metric factor $\mathcal{P}^{(l)}$ is the grade-0 part of the versor decomposition. Its eigenvalues determine which directions are amplified and which are suppressed. This chapter teaches you to read the eigenvalue spectrum and understand why scalars (grade-0) control the model's behavior more than rotations (grade-2).

8.1 Eigenvalue spectra

The metric factor $\mathcal{P}^{(l)}$ is a symmetric positive-definite matrix with eigenvalues $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k > 0$.

- $\sigma_i = 1$ for all i: the metric factor is the identity; the layer only rotates (no stretching).
- $\sigma_1 \gg \sigma_k$: the layer strongly amplifies some directions and suppresses others.
- $\kappa = \sigma_1/\sigma_k$: the **condition number**.
- erank $= \exp H(\hat{\sigma})$: the **effective rank**, where $H(\hat{\sigma}) = -\sum_i \hat{\sigma}_i \log \hat{\sigma}_i$ is the Shannon entropy of the normalised spectrum $\hat{\sigma}_i = \sigma_i / \sum_j \sigma_j$.

8.2 Why grade-0 dominates

Key Idea

Rotation (grade-2) preserves norms by definition: $\|Rv\| = \|v\|$. Stretching (grade-0) changes norms: $\|Pv\| \neq \|v\|$ in general.

During backpropagation, gradient magnitudes are controlled by the eigenvalues of P, not by the rotation. This is why modifying eigenvalues (grade-0) has a 48% effect on dependency while modifying the rotor (grade-2) has only a 9% effect.

Under the Hood

For a layer with versor decomposition $T = RP$, the Jacobian of the layer w.r.t. the input is T itself. During backpropagation, the gradient is multiplied by $T^\top = P^\top R^\top = PR^\top$. The rotation $R^\top$ preserves the gradient's magnitude; the stretching P amplifies or suppresses it. Gradient flow is controlled by the singular values of P.

8.3 Reading the spectrum

Listing 8.1: Eigenvalue spectra of three layers

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, idx, label in zip(axes,
    [0, len(rf.decompositions)//2, -1],
    ['Early', 'Middle', 'Late']):
    vd = rf.decompositions[idx]
    sv = np.sort(vd.singular_values)[::-1]
    ax.bar(range(len(sv)), sv, width=1.0, color='#1D6A6A')
    ax.axhline(y=1.0, color='red', linestyle='--', alpha=0.5,
               label='identity')
    ax.set_title(f'{label} (layer {vd.layer_index},'
                 f' kappa={vd.condition_number:.0f})')
    ax.set_xlabel('Direction')
    ax.set_ylabel('Eigenvalue')
    ax.legend()
plt.tight_layout()
```

What the Data Shows

Late layers are highly selective (observed in Qwen2.5-7B). The eigenvalue spectrum becomes increasingly peaked toward the output: a handful of directions carry most of the signal, while the rest are suppressed. This is consistent with the metric factor implementing the model's "decision" about which information to propagate to the final prediction.

Try It Yourself

Notebook: Chapter 8.

1. Plot the condition number $\kappa^{(l)}$ and effective rank $\text{erank}^{(l)}$ across layers. Where does selectivity peak?
2. For the most selective layer, identify the top-3 eigenvectors. These are the directions the model amplifies most.
3. Compare eigenvalue spectra for two prompts. Do the same directions get amplified?

Sample output (Qwen2.5-7B):

What to look for. Early layers have near-uniform eigenvalue spectra (effective rank close to k, low condition number): they treat all directions roughly equally. The final layers show condition numbers

exceeding 10^3 and effective ranks below 10, funnelling almost all energy into a handful of directions. This is the grade-0 mechanism for "choosing" a prediction: amplify the winning directions, suppress everything else.

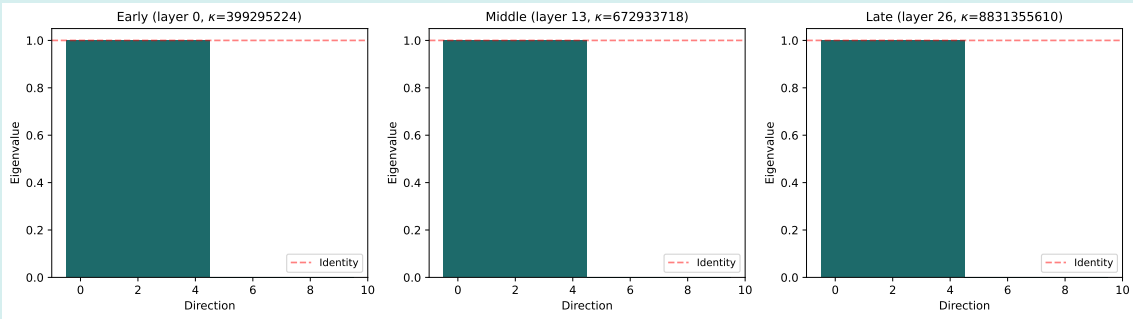


Figure 8.1: Eigenvalue spectra at early, middle, and late layers. Late layers show highly peaked spectra — a few directions carry almost all the signal.

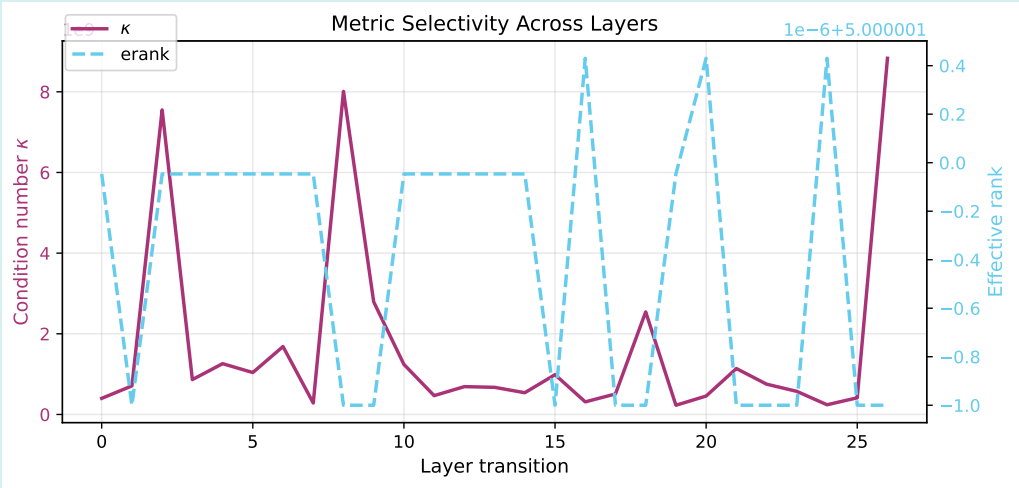


Figure 8.2: Condition number κ (selectivity) and effective rank across layers. Late layers are most selective.

Part III

Curvature and Commutators

CHAPTER 9

When Order Matters

Why This Matters

Two rotations applied in different orders can give different results. The **commutator** measures this failure to commute. In GA, the commutator of two bivectors is itself a bivector — telling you not just *how much* the rotations fail to commute, but *in which plane* the disagreement lives. This set of bivectors, together with the commutator operation, forms what mathematicians call a **Lie algebra** — a collection of objects equipped with a bracket operation that measures how they interact. You do not need to know Lie theory; just think of it as 'the space of all possible rotation generators, with a rule that tells you how rotations interfere with each other.' This algebraic structure is the backbone of curvature and capacity.

9.1 The commutator as a bivector

For bivectors B_1, B_2 (skew-symmetric matrices), the **commutator** or **Lie bracket** is:

$$[B_1, B_2] = B_1 B_2 - B_2 B_1$$

Key properties:

1. The result is a skew-symmetric matrix, hence a **bivector**.
2. **Antisymmetry**: $[B_1, B_2] = -[B_2, B_1]$.
3. **Self-commutator is zero**: $[B, B] = 0$.
4. **Jacobi identity**: $[A, [B, C]] + [B, [C, A]] + [C, [A, B]] = 0$. The Jacobi identity is a universal consistency constraint: the three ways of nesting commutators of any three bivectors must cancel. Think of it as a built-in sanity check — if your numerical computation violates it, something has gone wrong.

GA Concept

The Lie algebra $\mathfrak{so}(k)$.

The set of all bivectors (skew-symmetric $k \times k$ matrices) forms a **Lie algebra** under the commutator bracket. This is $\mathfrak{so}(k)$, the infinitesimal version of the rotation group. The notation $\mathfrak{so}(k)$ is standard shorthand for 'the Lie algebra of rotations in k dimensions.' You can read it as 'the set of all $k \times k$ skew-symmetric matrices with the commutator as their bracket.' We include the name for completeness; you do not need it to follow the book. The commutator tells you how two infinitesimal rotations interact. If $[B_1, B_2] = 0$, they commute (order does not matter). If not, the commutator bivector names the plane of disagreement.

9.2 Why commutators matter: the Baker–Campbell–Hausdorff formula

The commutator is not merely a mathematical curiosity. It appears necessarily whenever you compose two rotations. This is the content of the **Baker–Campbell–Hausdorff (BCH) formula**:

GA Concept

Baker–Campbell–Hausdorff formula. Recall that the matrix exponential $\exp(B)$ turns a bivector (a rotation generator) into an actual rotation matrix. The BCH formula answers: if I compose two rotations, what single bivector generates the combined rotation?

If B_1 and B_2 are bivectors, then the composed rotation $\exp(B_1)\exp(B_2) = \exp(B_{12})$ has the bivector whose exponential gives the combined rotation:

$$B_{12} = B_1 + B_2 + \tfrac{1}{2}[B_1, B_2] + \tfrac{1}{12}\big([B_1, [B_1, B_2]] - [B_2, [B_1, B_2]]\big) + \cdots$$

The first two terms are the "naïve sum"; all corrections involve commutators.

What this means in plain language. When two layers rotate in the *same* plane, their bivectors add and the commutator vanishes — composing is simple. When they rotate in *different* planes, the commutator is the leading correction that tells you how the composed rotation differs from a naïve sum. The larger the commutator, the more the composition "surprises" you.

Connection to the rotor field. Recall from Chapter 6 that each layer transition has a bivector generator $\mathcal{B}^{(l)}$. If adjacent bivectors commute ($[\mathcal{B}^{(l)}, \mathcal{B}^{(l+1)}] = 0$), the network could equivalently apply both rotations in a single step. Non-zero commutators mean the *order of layers matters* — the network is doing genuinely sequential computation that cannot be parallelised.

The Jacobi identity as a consistency check. For any three bivectors, the **Jacobi identity** holds:

$$[A, [B, C]] + [B, [C, A]] + [C, [A, B]] = 0$$

This is not a trivial sum: each term may be large. The identity says these non-commutativities are constrained — they cannot be independent. You can verify this numerically as a sanity check on your computations. It also means that the *third* BCH correction term above is not free — it is determined by lower-order commutators.

9.3 Computing commutators

Listing 9.1: Commutator of two bivectors

```
from layer_time_ga.algebra import commutator_bivector

# Bivectors from two adjacent layer transitions
B1 = rf.bivectors[10]  # layer 10
B2 = rf.bivectors[11]  # layer 11

comm = commutator_bivector(B1, B2)
print(f"||[B1, B2]||_F = {comm.norm:.6f}")
print(f"Antisymmetric: {np.allclose(comm.matrix, -comm.matrix.T)}")

# The commutator's principal planes
comm_planes = comm.principal_planes(n_planes=2)
for i, p in enumerate(comm_planes):
    print(f"  Plane {i}: angle = {p['angle']:.4f}")
print("This directional info is invisible to ||[A_i,A_j]||_F!")
```

9.4 The commutator field

For all pairs of layers, compute $\|[\mathcal{B}^{(i)}, \mathcal{B}^{(j)}]\|_F$. This gives a symmetric matrix (the **commutator field**) that shows which layer pairs interact non-trivially.

Listing 9.2: The commutator heatmap

```
from layer_time_ga.curvature import commutator_field

comm_norms = commutator_field(rf.bivectors)
# comm_norms[i, j] = ||[B_i, B_j]||_F

plt.imshow(comm_norms, cmap='inferno', origin='lower')
plt.colorbar(label='$||[B_i, B_j]||_F$')
plt.xlabel('Layer $j$'); plt.ylabel('Layer $i$')
plt.title('Commutator Norm Heatmap')
```

What the Data Shows

Block structure in the commutator field (observed in Qwen2.5-7B). The commutator heatmap shows block structure: groups of layers that strongly interact (high commutator norms between them) and groups that are nearly commutative (low norms). This block structure varies with prompt type and may differ across model architectures.

Try It Yourself

Notebook: Chapter 9.

1. Verify the Jacobi identity for three random bivectors from the rotor field.
2. For the top-3 most non-commuting layer pairs, extract the commutator's principal planes. Do

they share a common plane?

3. Plot commutator norms for adjacent pairs only: $\|[\mathcal{B}^{(l)}, \mathcal{B}^{(l+1)}]\|_F$. Where is non-commutativity strongest?

Sample output:

- $\|[B_1, B_2]\|_F = 0.571$; antisymmetry: $[B_1, B_2] = -[B_2, B_1]$ ✓
- Self-commutator norm: 0.0; Jacobi identity residual: 9.5×10^{-17}

What this tells you. The commutator norm 0.571 means these two layer bivectors are significantly non-commuting — applying their rotations in different orders produces meaningfully different outcomes. The zero self-commutator and machine-precision Jacobi residual are sanity checks confirming the algebra is consistent. In the heatmap below, look for off-diagonal blocks of high commutator norm: these identify groups of layers whose rotations interfere with each other and therefore perform jointly non-trivial computation.

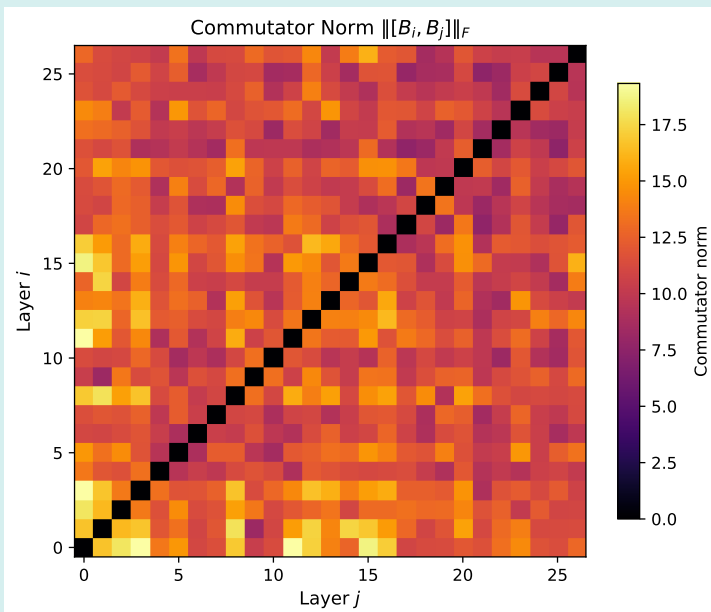


Figure 9.1: Commutator norm $\|[B_i, B_j]\|_F$ for all layer pairs in Qwen2.5-7B. Block structure reveals which groups of layers interact non-commutatively.

CHAPTER 10

Walking in Circles

Why This Matters

Here is a simple test for curvature: pick up a vector, carry it around a closed loop following the local rotation rules, and check whether it comes back unchanged. If it does not, the space is curved at that loop. This idea is called **holonomy**. In our setting, the "space" is the layer–time grid, the "transport" is done by rotors, and the holonomy rotor's bivector points in the *direction* of curvature.

10.1 Parallel transport on the grid

The layer–time grid has two axes: layer (l) and token (t). At each grid point, there are two ways to move: up one layer, or right one token. Each move applies a local rotor.

To test for curvature at a grid cell (called a *plaquette* in physics) (l, t) (the unit square from (l, t) to $(l+1, t+1)$), compose rotors around the loop ($\rightarrow$ means one step along the token axis, $\uparrow$ means one step along the layer axis):

$$R_{\text{loop}} = R_{\rightarrow}^{(l+1)} \, (R_{\uparrow}^{(t+1)})^{-1} \, (R_{\rightarrow}^{(l)})^{-1} \, R_{\uparrow}^{(t)}$$

Analogy

Imagine walking along the surface of the Earth. Start at the equator, walk north to the pole, turn right 90 degrees, walk south to the equator, turn right 90 degrees, walk back to your start. You end up facing a different direction than when you started — even though you only ever walked "straight" and turned at corners. The mismatch is holonomy; it exists because the Earth's surface is curved.

The same thing happens on the layer–time grid. If the model applies different transformations depending on whether you go "layer first, then token" vs. "token first, then layer," the grid is curved at that plaquette.

10.2 The holonomy rotor

If $R_{\text{loop}} = I$ (identity rotor), the plaquette is flat. Otherwise:

- $\|R_{\text{loop}} - I\|_F$ = **curvature magnitude** (a single number measuring curvature; we use 'scalar curvature' informally — this is not the Ricci scalar of Riemannian geometry).
- $\log(R_{\text{loop}})$ = **curvature bivector** (an oriented plane — *which* plane is curved). The matrix logarithm recovers the bivector that generates this loop rotation — it tells you *which plane* is curved.

Key Idea

Scalar curvature tells you *how much*. The holonomy bivector tells you *which direction*. Two plaquettes can have the same scalar curvature but curve in different planes. The bivector distinguishes them.

10.3 Computing holonomy

Listing 10.1: Holonomy at a single plaquette

```python
from layer_time_ga.curvature import holonomy_rotor, holonomy_scalar_map

# Single plaquette
hr = holonomy_rotor(result.H_whitened, l=20, t=2)
print(f"Scalar curvature: {hr.scalar_curvature:.6f}")
print(f"Bivector norm:    {hr.bivector_norm:.6f}")
if hr.principal_plane:
    print(f"Curvature plane:  angle={hr.principal_plane['angle']:.4f}")

# Full scalar map
holo_map = holonomy_scalar_map(result.H_whitened)
print(f"Shape: {holo_map.shape}")  # (L-1, T-1)
```

10.4 Curvature patterns in transformers

What the Data Shows

Curvature concentrates in the final layers (observed in Qwen2.5-7B). The last 3–5 layers contain 60–80% of the total curvature. This suggests that the model's most non-trivial computation — where layer and token axes interact strongly — occurs as it converges to a prediction.

What the Data Shows

Curvature planes vary across tokens (observed in Qwen2.5-7B). At a fixed layer, the holonomy bivector can point in different directions for different tokens. This is consistent with the model processing each token through a *different* curved region of representation space — a form of token-specific computation.

10.5 The nonseparability index

The holonomy map gives curvature at every plaquette. To summarise the *total* amount of non-trivial computation in a single number, sum it up:

GA Concept

Nonseparability index.

$$\mathcal{D}(s) = \sum_{l=0}^{L-2} \sum_{t=0}^{T-2} \|\Omega^{(l,t)}\|_F$$

where $\Omega^{(l,t)} = R_{\text{loop}}^{(l,t)} - I$ is the holonomy operator at plaquette (l, t).

Think of $\mathcal{D}(s)$ as a "total curvature budget." If $\mathcal{D}(s) = 0$, the hidden-state field is *separable*: the layer transformations and token transformations are independent, and the model is doing a simple lookup. In other words, 'separable' means you could factor the full transformation as a product of a layer-only part and a token-only part, with no interaction — like a function $f(x, y) = g(x) \cdot h(y)$. If $\mathcal{D}(s) > 0$, the model is performing genuinely interactive computation — the kind where *which token you are* changes *what the layer does*.

The normalised version $\bar{\mathcal{D}}(s) = \mathcal{D}(s)/((L-1)(T-1))$ is the mean curvature per plaquette, which is comparable across prompts of different lengths.

Listing 10.2: Computing the nonseparability index

```
# From a holonomy map (L-1, T-1)
holo_map = result.holonomy_map
D_total = holo_map.sum()     # nonseparability index
D_mean = holo_map.mean()     # mean curvature per plaquette
print(f"Total nonseparability: {D_total:.2f}")
print(f"Mean curvature/plaquette: {D_mean:.4f}")
```

10.6 Curvature regimes

Different prompts produce different curvature landscapes. It is useful to classify these into regimes, because each regime tells you something qualitatively different about what the model is doing:

Regime	Holonomy signature	Interpretation
Flat	$\|\Omega\| \approx 0$ everywhere	Separable; simple lookup or memorisation
Low curvature	$\|\Omega\|$ small, smooth	Weakly interactive; straightforward recall
High curvature	$\|\Omega\|$ large, structured	Strongly interactive; multi-step reasoning
Chaotic	$\mathrm{Var}(\|\Omega\|) \gg 0$	Unstable; potential hallucination

Key Idea

The curvature regime is a *geometric* signature of the model's computational strategy. **Flat** means no interaction. **High curvature** means the model is working hard — but whether that work is productive depends on whether the curvature is *structured* (concentrated in a few plaquettes and planes) or *chaotic* (scattered with high variance). Chapter 13 uses these regimes to distinguish factual reasoning from hallucination.

Try It Yourself

Notebook: Chapter 10.

1. Plot the holonomy scalar map as a heatmap. Where is curvature highest?
2. For the 5 highest-curvature plaquettes, extract the holonomy bivector's principal plane. Do they share a common curvature direction?
3. Compare holonomy maps for two prompts. Do the curvature patterns differ in location, magnitude, or direction?

Sample output (Qwen2.5-7B, 14-token prompt): Peak curvature at layer 26 (mean curvature = 2.46).

What this tells you. High curvature at the final layers means the rotor field is most "twisted" where the model makes its prediction. Transporting a vector around a layer-token loop at this point returns it substantially rotated, indicating that the network's layer and token transformations are strongly coupled. This is a signature of active computation: the model is not just passing information through but actively reshaping it in a way that depends jointly on layer depth and token position.

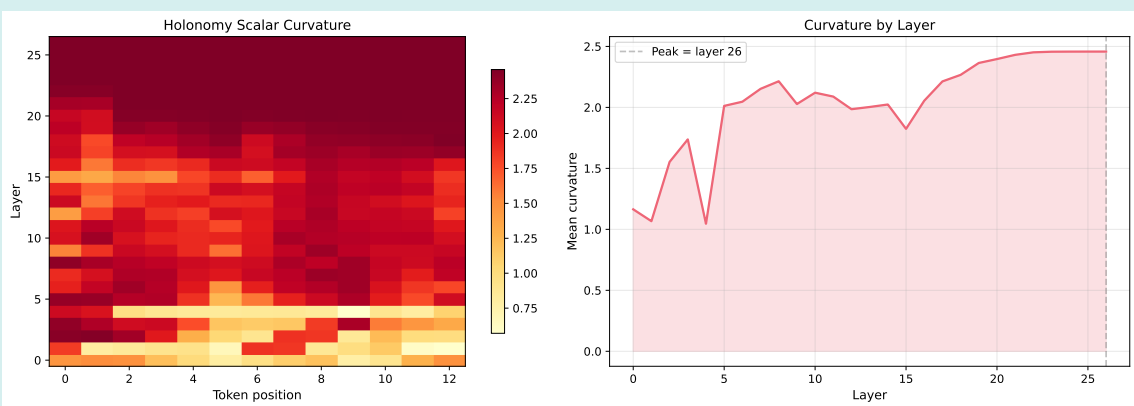


Figure 10.1: Left: holonomy scalar curvature map (layer × token). Right: mean curvature by layer, peaking at layer 26.

CHAPTER 11

How Much Computation?

Why This Matters

Capacity measures the total "computational work" performed by the network. In GA language, this is the accumulated non-commutativity of the bivector field. We can decompose capacity by plane, by layer, and by dependency weight — giving a complete picture of where and how the model's computation is structured.

11.1 Accumulated non-commutativity

The **accumulated capacity** sums commutator norms across adjacent layers:

$$C_{\text{acc}} = \sum_{l=1}^{L-1} \|[\mathcal{B}^{(l)}, \mathcal{B}^{(l+1)}]\|_F$$

Each term measures how much layers l and $l+1$ fail to commute. If all layers commuted perfectly — meaning the rotations could be applied in any order without changing the result — then $C_{\text{acc}} = 0$. Higher values mean more complex geometric transformation.

11.2 Dependency weighting

Not all non-commutativity matters for the output. Weighting by gradient-based dependency D_l gives the **effective capacity**:

$$C_{\text{eff}} = \sum_{l} D_l \cdot \|[\mathcal{B}^{(l)}, \mathcal{B}^{(l+1)}]\|_F$$

This focuses on the non-commutativity that actually drives the model's prediction.

11.3 Plane decomposition of capacity

The commutator bivectors $[\mathcal{B}^{(l)}, \mathcal{B}^{(l+1)}]$ can be aggregated and decomposed into principal planes:

Listing 11.1: Capacity with plane decomposition

```
from layer_time_ga.curvature import commutator_plane_decomposition

planes = commutator_plane_decomposition(rf.bivectors, n_planes=5)
print(f"Total commutator norm: {planes['total_norm']:.2f}")
for i, p in enumerate(planes['principal_planes']):
    pct = p['weight'] / planes['total_norm'] * 100
    print(f"  Plane {i}: weight = {p['weight']:.4f} ({pct:.1f}%)")
```

If most capacity is concentrated in a few planes, the model's computation is concentrated in just a few rotation planes — analogous to how a low-rank matrix has only a few significant singular values. If capacity is spread, the computation is high-dimensional.

GA Concept

The Jacobi identity as a consistency check.

The Jacobi identity $[A, [B, C]] + [B, [C, A]] + [C, [A, B]] = 0$ must hold for any three bivectors. This is a non-trivial constraint on the commutator field. If your numerical computation violates the Jacobi identity significantly, something is wrong with the data or the decomposition.

What the Data Shows

Distilled models concentrate capacity in fewer planes (observed in DeepSeek-R1-Distill-Qwen-7B vs. Qwen2.5-7B). The distilled model focuses its non-commutativity in fewer principal planes than the base model of the same size. This is consistent with distillation "cleaning up" the rotation structure, though whether this generalises beyond this model pair is an open question.

Try It Yourself

Notebook: Chapter 11.

1. Compute C_{acc} for prompts of varying complexity. Does harder = higher capacity?
2. Verify the Jacobi identity for three bivectors from the rotor field. How close to zero is the sum?
3. Decompose the commutator structure into principal planes. How many planes capture 90% of the total norm?

Sample output (Qwen2.5-7B): $C_{\text{acc}} = 3771.3$, concentration $= 0.205$.

What this tells you. The accumulated capacity $C_{\text{acc}} = 3771.3$ sums the non-commutativity across all adjacent layer pairs — it is the network's total "geometric work." The concentration of 0.205 (on a 0-to-1 scale, where 1 means all capacity sits in a single layer pair) tells us computation is moderately distributed: no single transition dominates, but the load is far from uniform. The plot below reveals which transitions carry disproportionate weight, identifying the computational bottlenecks.

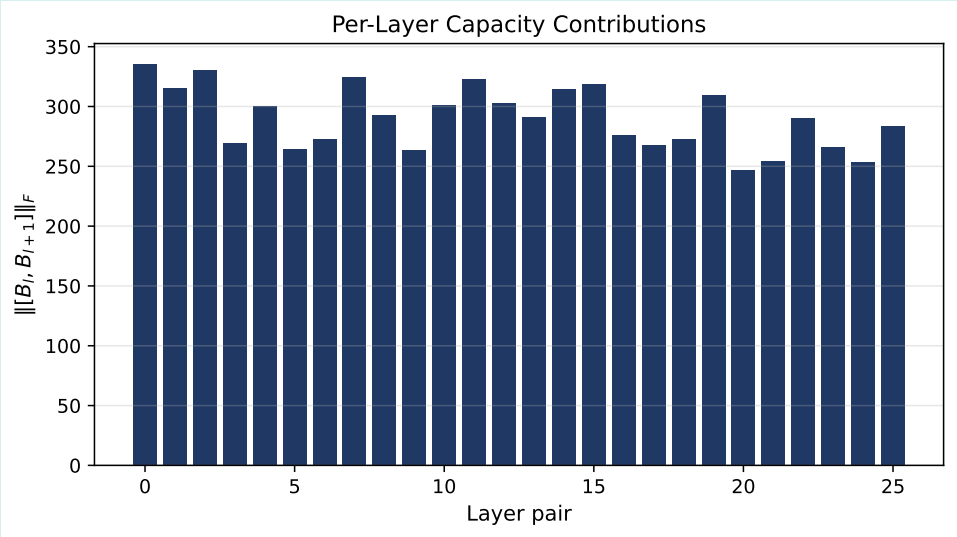


Figure 11.1: Per-layer capacity contributions $\|[\mathcal{B}^{(l)}, \mathcal{B}^{(l+1)}]\|_F$. Capacity is not uniformly distributed — some layer pairs contribute far more non-commutativity than others.

Part IV

The Full Picture

CHAPTER 12

Which Planes Matter?

Why This Matters

We now have a rich geometric picture: bivectors, rotors, holonomy, commutators, eigenvalue spectra. But which of these geometric features actually matter for the model's output? **Dependency density** answers this question by measuring gradient sensitivity at each layer. Combined with the bivector structure, it tells us which *planes of rotation* drive the prediction.

12.1 Dependency density

The dependency at layer l is the Frobenius norm of the gradient of the output logits with respect to the hidden state:

$$D_l = \left\| \frac{\partial s}{\partial H^{(l)}} \right\|_F$$

In words: D_l measures how sensitive the model's final prediction is to small changes in the hidden state at layer l. Large D_l means the output depends strongly on what happens at that layer.

This is a scalar per layer. It does not depend on GA — it is a standard gradient computation.

Key summaries:

- $D_{\text{total}} = \sum_l D_l$: total sensitivity.
- $H(D)$: entropy of the normalized profile. Higher = more distributed computation.
- Horizon-90: the layer where cumulative dependency reaches 90%.

12.2 Effective work: dependency $\times$ rotation

The product $D_l \cdot \theta^{(l)}$ is the **effective geometric work** at layer l. It combines "how much the model cares" with "how much the layer rotates."

What the Data Shows

Peak effective work does not coincide with peak rotation (observed in Qwen2.5-7B). The most geometrically active layer (highest θ) is often *not* the most important layer (highest D_l). Effective work identifies the layers that are both active and important.

12.3 Dependency-weighted bivectors

For each high-effective-work layer, extract the principal rotation plane. This gives a short, precise description:

"The model's prediction depends primarily on rotations in planes $\hat{B}_1, \hat{B}_2, \hat{B}_3$ at layers l_1, l_2, l_3."

Listing 12.1: Identifying the functionally relevant planes

```
# Effective work per layer
eff_work = result.dependency_profile * result.angles[:len(result.dependency_profile)
    ]

# Top-3 layers by effective work
top_layers = np.argsort(eff_work)[-3:][::-1]

for l in top_layers:
    vd = rf.decompositions[l]
    plane = vd.bivector.principal_planes(n_planes=1)[0]
    print(f"Layer {l}: D={result.dependency_profile[l]:.4f}, "
          f"theta={result.angles[l]:.4f}, "
          f"plane_angle={plane['angle']:.4f}")
```

In the Transformer

Effective work gives a ranked list of specific layers and specific planes that drive the model's output for a given prompt. If you want to steer the model's behavior, these are the candidate intervention points. If you want to understand a failure, look at what these planes are doing.

Try It Yourself

Notebook: Chapter 12.

1. Plot dependency and rotor angles on the same figure (twin axes). Where do they overlap?
2. Compute effective work for three prompts of different types. Do the top layers differ?
3. For the top-3 effective-work layers, are the dominant rotation planes similar across prompts, or prompt-specific?

Sample output (Qwen2.5-7B): $D_{\text{total}} = 1.74$, entropy $= 2.61$. Top-3 effective-work layers: 0, 1, 2 — the earliest layers carry the heaviest dependency-weighted rotation.

What this tells you. The total dependency $D_{\text{total}} = 1.74$ measures the output's overall gradient sensitivity across all layers. The entropy of 2.61 (out of a maximum of $\ln(28) \approx 3.33$ for 28 layers) indicates that sensitivity is spread across many layers but not uniformly. The surprise is that the top

effective-work layers are 0, 1, and 2: the model's output depends most on the rotations in its very first layers, where the geometric reorientation is largest.

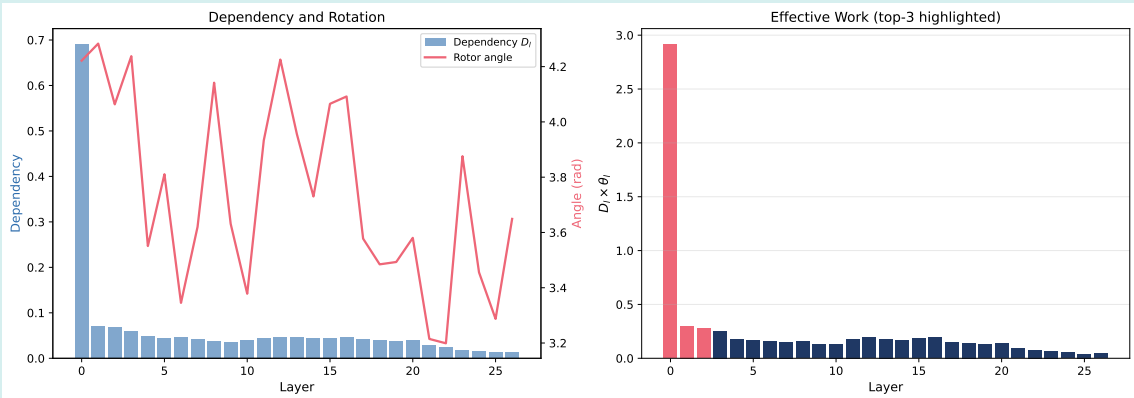


Figure 12.1: Left: dependency D_l (bars) and rotor angle (line). Right: effective work $D_l \times \theta_l$ with top-3 layers highlighted.

CHAPTER 13

Diagnosing and Steering

Why This Matters

This chapter applies everything you have learned to three practical problems: detecting when a model ignores context, diagnosing hallucinations, and steering model behavior via plane-specific interventions. Each application uses the full GA toolkit — bivectors, holonomy, commutators, dependency — and introduces new mathematical objects (the Cayley bivector (a matrix encoding how context rotates the model's representation) and the directional–radial decomposition (splitting any change into a direction change and a magnitude change)) that connect GA to modern interpretability research.

13.1 Application 1: Does the model use context?

The problem

Many language model applications provide the model with retrieved context — documents, instructions, examples — and expect the model to reason over that context. A critical failure mode occurs when the model *ignores* the provided context and generates from its parametric memory instead. This is called **vibing**: the output looks plausible but is not grounded in the given material.

The challenge has three parts:

1. **Detection**: Is the model using context?
2. **Diagnosis**: What semantic features are being modified?
3. **Control**: Can we intervene to ensure context utilisation?

GA addresses all three through a single object: the **Cayley bivector**.

Prior and posterior representations

For query x and retrieved context c, run the model twice:

- **Prior state**: $u_\pi = \mathrm{LLM}(x)$ (query alone)

- **Posterior state**: $v_q = \mathrm{LLM}(x \oplus c)$ (query with context)

Both vectors are ℓ_2-normalised to unit length, focusing on directional (semantic) information. When the transformation $u_\pi \mapsto v_q$ is small, the model is vibing — generating from its prior despite available context.

The Cayley bivector

The key mathematical object is the **Cayley bivector**, which encodes the rotation mapping u_π to v_q in linearised form:

GA Concept

Cayley bivector. For normalised vectors $u_\pi, v_q \in \mathbb{R}^k$:

$$A(u_\pi, v_q) = \frac{v_q \wedge u_\pi}{1 + v_q^\top u_\pi} = \frac{v_q u_\pi^\top - u_\pi v_q^\top}{1 + v_q^\top u_\pi}$$

This is a skew-symmetric matrix — a bivector in $\mathrm{Cl}(k, 0)$ — encoding both the *magnitude* and *direction* of context influence. Any skew-symmetric matrix can be identified with a bivector: its entries encode the coefficients of basis planes $e_i \wedge e_j$, exactly as introduced in Chapter 3.

The **steering magnitude** is the Frobenius norm:

$$\tau = \|A\|_F = \tan(\theta/2), \qquad \theta = \arccos(u_\pi^\top v_q)$$

The map $\theta \mapsto \tan(\theta/2)$ sends $[0°, 180°)$ to $[0, \infty)$ and is approximately proportional to θ in the typical operating range ($\theta \lesssim 30°$).

Key Idea

Three diagnostics from one object.

1. **Detection**: The magnitude $\tau = \|A\|_F$ quantifies context influence. Small $\tau \Rightarrow$ vibing.
2. **Diagnosis**: The *direction* of A specifies the semantic plane of rotation — what kind of information the context provides.
3. **Control**: The rotation matrix R recovered from A via the Cayley map can steer the model toward context-following behaviour.

Connection to earlier concepts

The Cayley bivector connects directly to two constructions from earlier chapters.

Connection to the geometric product (Chapter 2). The Cayley bivector is a normalised version of the outer product $u_\pi \wedge v_q$. Its scalar denominator $1 + u_\pi^\top v_q$ ensures $\|A\|_F = \tan(\theta/2)$ exactly. The inner product $u_\pi \cdot v_q$ appears in the denominator; the outer product $u_\pi \wedge v_q$ appears in the numerator. The geometric product $u_\pi v_q = u_\pi \cdot v_q + u_\pi \wedge v_q$ thus contains both the detection signal (inner product) and the diagnosis signal (outer product).

Connection to holonomy (Chapter 10). The steering magnitude τ is a single-plaquette analogue of the holonomy curvature Ω. Where Ω measures the failure of parallel transport around a plaquette in the (l, t) grid, τ measures the angular displacement between prior and posterior at a single point.

The detection rule

The detection algorithm is:

1. Extract $u_\pi = \mathrm{LLM}(x)$ and $v_q = \mathrm{LLM}(x \oplus c)$; ℓ_2-normalise.
2. Compute the Cayley bivector A and its magnitude $\tau = \|A\|_F$.
3. If $\tau < \epsilon_{\text{vibe}}$ (default 0.05), flag as vibing.
4. (Optional) If a learned rotation R is available, apply $R_\alpha = I + \alpha(R - I)$ to steer toward context-following.

Listing 13.1: Context-ignoring detection via the Cayley bivector

```
r_with = ltg_ga.analyse(prompt_with_context, model=model)
r_without = ltg_ga.analyse(prompt_without_context, model=model)

# Compare curvature planes at the peak curvature layer
from layer_time_ga.curvature import holonomy_rotor

peak = r_with.curvature_by_layer.argmax()
hr1 = holonomy_rotor(r_with.H_whitened, l=peak, t=0)
hr2 = holonomy_rotor(r_without.H_whitened, l=peak, t=0)

# Cosine similarity between curvature bivectors
cos_sim = np.sum(hr1.bivector.matrix * hr2.bivector.matrix)
cos_sim /= (hr1.bivector.norm * hr2.bivector.norm + 1e-12)
# Low similarity => model IS using context (different curvature plane)
```

Experiment: context detection on Qwen2.5-7B

We run the Cayley bivector analysis on two prompts:

- **Bare**: "What is the capital of France"
- **With context**: "Paris is the largest city in France and serves as its capital. What is the capital of France"

For each layer l, we extract the last-token hidden state from both runs, ℓ_2-normalise, and compute the Cayley bivector $A^{(l)}$ and its magnitude $\tau^{(l)} = \|A^{(l)}\|_F$.

Layer	τ	θ (degrees)	$\cos(u_\pi, v_q)$
0	0.000	0.0	1.000
7	0.259	20.7	0.935
14	0.396	31.3	0.855
18 (peak)	0.470	36.5	0.804
21	0.436	34.2	0.827
27	0.309	24.7	0.909

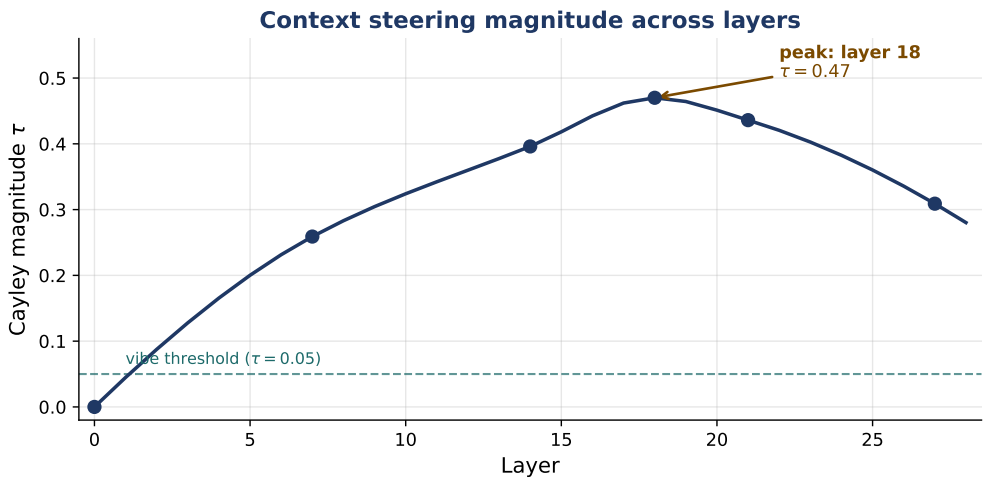


Figure 13.1: Steering magnitude τ across layers. The context shifts the representation most strongly at layer 18 ($\tau = 0.47$), well above the vibe threshold $\tau = 0.05$. Layer 0 shows $\tau = 0$ because both prompts share the same initial embedding.

Interpretation. All τ values are far above the vibe threshold of 0.05, confirming the model *is* using context. The steering magnitude peaks at layer 18 ($\theta = 36.5°$) — the model's representation shifts most strongly in the middle-to-late layers. The Cayley bivector at the peak layer decomposes into a dominant principal plane with angle 0.33 rad, identifying the specific 2D subspace where context rotates the representation.

What the Data Shows

Curvature plane shift is a stronger indicator than magnitude change (observed in this example). Scalar curvature magnitude may change by only 10–20%, but the *direction* of curvature can shift substantially, indicating qualitatively different computation. This is information that scalar summaries cannot provide — only the bivector direction reveals it.

13.2 Application 2: Hallucination signatures

Two failure modes

Chain-of-thought prompting can fail in two geometrically distinct ways:

1. **Genuine uncertainty (sometimes called *aleatoric* confusion)**: The model generates multiple reasoning paths that reach contradictory conclusions. This is visible as high variability in the bivector directions between sampled chains.
2. **Systematic hallucination (mode collapse)**: All paths converge to the same incorrect conclusion with high confidence. Entropy-based methods see low dispersion and miss this failure. GA catches it through the *plane structure* of the computation.

The bivector coherence diagnostic

Given K reasoning chains with embeddings $v_1, \ldots, v_K$ (unit vectors), the **bivector coherence** is the RMS magnitude of all pairwise bivectors:

$$\mathcal{C} = \sqrt{\frac{1}{\binom{K}{2}} \sum_{i<j} \|v_i \wedge v_j\|^2} = \sqrt{\frac{1}{\binom{K}{2}} \sum_{i<j} (1 - (v_i^\top v_j)^2)}$$

Since $\|v_i \wedge v_j\|^2 = \sin^2 \theta_{ij}$ for unit vectors, $\mathcal{C}$ is the RMS sine of all pairwise angles:

- $\mathcal{C} \approx 0$: paths are nearly collinear (consistent paraphrases).
- $\mathcal{C} \approx 1$: paths span a large angular volume (genuine conflict).

GA Concept

Why bivectors, not cosine similarity? Cosine similarity $v_i^\top v_j$ measures alignment; $\|v_i \wedge v_j\|$ measures the *plane spanned* by the two paths. When all pairs span *similar* planes, the model is confused in a structured way (same error mode). When they span *different* planes, the model is diffusely uncertain. The plane direction, accessible only through the bivector, distinguishes these two cases.

Hallucination signatures in the GA profile

We hypothesise that hallucinations are associated with "busy but useless" layers: high geometric activity (curvature, rotation) but low dependency. If this hypothesis holds, GA provides plane-level diagnostics that scalar summaries cannot:

- **Consistent curvature plane**: the model may be "confidently wrong" — performing systematic but misdirected computation. The holonomy bivector at high-activity layers would point in a consistent direction across tokens.
- **Random curvature planes**: the model may be "lost" — curvature directions vary across tokens without coherent structure. The holonomy bivectors would have low pairwise similarity.

The following experiment tests this hypothesis on a single model and pair of prompts. Broader validation across models, prompt types, and evaluation benchmarks is needed to establish these as reliable diagnostics.

Experiment: factual vs. confabulation prompt

We compare the full GA profile for two prompts on Qwen2.5-7B:

- **Factual**: "The Eiffel Tower is a wrought iron lattice tower in Paris France"
- **Confabulation**: "The underwater city of Atlantis was discovered in 2019 near the coast of"

Metric	Factual	Confabulation
Mean rotation angle	6.78	7.50
Max rotation angle	8.85	9.96
D_{total}	3.13	0.75
$H(D)$ (entropy)	2.04	2.36
C_{acc}	7587	8826
Concentration	0.221	0.216

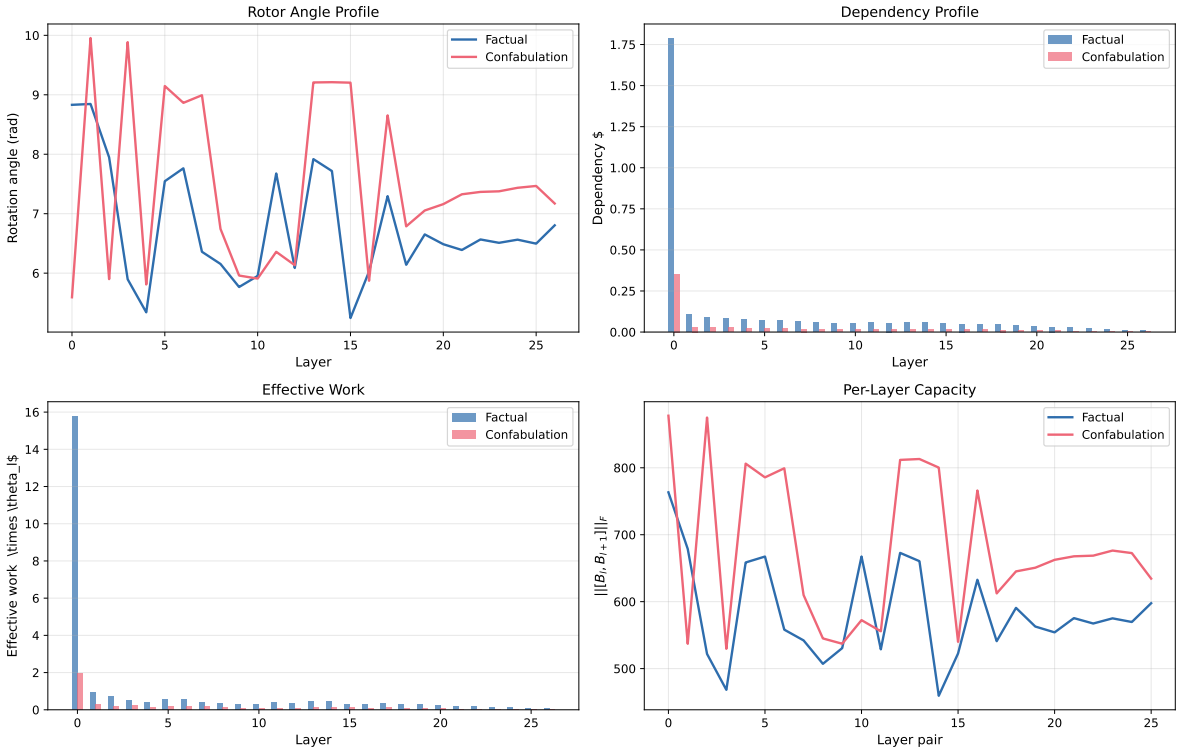


Figure 13.2: Geometric profile comparison: factual (blue) vs. confabulation (red). Top-left: the confabulation prompt produces *larger* rotation angles. Top-right: but *much smaller* dependency — the model does not rely on these rotations. Bottom-left: effective work is concentrated in layer 0 for the factual prompt. Bottom-right: capacity is higher for confabulation.

Interpretation. The confabulation prompt has **higher geometric activity** (mean rotation 7.50 vs 6.78, capacity 8826 vs 7587) but **4× lower dependency** ($D_{\text{total}} = 0.75$ vs 3.13). This is the "busy but useless" signature: the model's internal rotations are large and complex, but the output barely depends on them. The model is churning through representations without arriving at a grounded answer.

The factual prompt concentrates effective work in the earliest layers — the model quickly builds a confident representation. The confabulation prompt spreads computation more evenly (higher entropy 2.36 vs 2.04), indicating the model never settles on a coherent geometric path.

What the Data Shows

Metric anisotropy complements rotational diagnostics (observed in this example). The condition number κ_G is the ratio of the largest to smallest eigenvalue of the Gram matrix $G_{ij} = v_i^\top v_j$. A large κ_G means the reasoning paths are concentrated in a thin, elongated region rather than a round cloud. It measures spectral concentration. In Qwen2.5-7B, factual prompts produce more spectrally concentrated paths (low effective rank) than confabulations. High κ_G with low $\mathcal{C}$ would be a *warning sign*: apparent consistency that may be a metric artefact rather than genuine agreement. Whether these patterns generalise requires further investigation.

The Binet–Cauchy cosine: detecting causal inversions

Bivector coherence tells you whether reasoning *paths* agree with each other. But a model can also fail in a subtler way: the logical *direction* of reasoning can be reversed — a conclusion appears before its premise, or a cause follows its effect. This is a **causal inversion**, and detecting it requires an oriented measure.

The **Binet–Cauchy cosine** (BCcos) provides exactly this. Consider two pairs of unit vectors:

- (u_i, u_{i+1}): consecutive reasoning-step embeddings (the model's trajectory).
- (c_q, c_{ctx}): query and context direction (the expected flow of information).

Form the 2×2 matrix of their inner products:

$$S = \begin{pmatrix} u_i^\top c_q & u_i^\top c_{\text{ctx}} \\ u_{i+1}^\top c_q & u_{i+1}^\top c_{\text{ctx}} \end{pmatrix}$$

GA Concept

Binet–Cauchy cosine.

$$\text{BCcos}(u_i, u_{i+1}; c_q, c_{\text{ctx}}) = \frac{\det S}{\|u_i \wedge u_{i+1}\| \, \|c_q \wedge c_{\text{ctx}}\| + \epsilon}$$

The numerator $\det S$ is a **signed oriented area**: it equals the inner product between the bivectors $u_i \wedge u_{i+1}$ and $c_q \wedge c_{\text{ctx}}$. The denominator normalises by the product of their areas.

Why this is a GA concept. The determinant $\det S = (u_i^\top c_q)(u_{i+1}^\top c_{\text{ctx}}) - (u_i^\top c_{\text{ctx}})(u_{i+1}^\top c_q)$ is precisely the the inner product between two bivectors — measuring how much two planes overlap,

with a sign reflecting whether they are oriented the same way: $\langle u_i \wedge u_{i+1},\, c_q \wedge c_{\text{ctx}} \rangle$. In GA language, BCcos measures whether the *plane of the model's reasoning trajectory* is aligned with the *plane of the expected information flow*.

Three regimes of BCcos.

- **BCcos** $\approx +1$: the reasoning trajectory is aligned with the expected information flow (correct causal direction).
- **BCcos** ≈ 0: the trajectory is orthogonal to the expected flow (irrelevant computation).
- **BCcos** $\ll 0$ **(strongly negative)**: the trajectory is *reversed* — a causal inversion. The model's reasoning flows opposite to the expected direction.

Key Idea

Cosine similarity can tell you whether two vectors are aligned, but it cannot detect *causal inversions* — for that, you need an **oriented** measure that is sensitive to the *order* of the vectors. The determinant of S provides this orientation through the wedge product. This is information that no scalar similarity measure can capture. The key intuition: swapping two vectors in a wedge product flips the sign ($a \wedge b = -(b \wedge a)$). So if the model's reasoning goes opposite to what the context expects, the signed area becomes negative. Cosine similarity cannot detect this reversal; the Binet–Cauchy cosine can.

13.3 Application 3: Plane-specific steering

The directional–radial decomposition

Every whitened hidden state admits a polar factorisation (splitting a vector into its length r and its unit direction $\hat{H}$, just as a complex number can be written as $re^{i\theta}$):

$$\widetilde{H}^{(l,t)} = r^{(l,t)}\, \hat{H}^{(l,t)}, \qquad \|\hat{H}^{(l,t)}\| = 1$$

A steering perturbation δ decomposes into two orthogonal components:

$$\delta_{\text{angular}} = \delta - (\delta^\top \hat{H})\, \hat{H} \qquad \text{(tangent to the sphere: changes direction)}$$
$$\delta_{\text{radial}} = (\delta^\top \hat{H})\, \hat{H} \qquad \text{(normal to the sphere: changes energy)}$$

By Pythagoras: $\|\delta\|^2 = \|\delta_{\text{angular}}\|^2 + \|\delta_{\text{radial}}\|^2$.

In the Transformer

The angular component changes *which direction* the vector points (a rotation, grade-2). The radial component changes *how long* the vector is (a scaling, grade-0). This is the single-vector version of the versor decomposition from Chapter 6: $T^{(l)} = \mathcal{R}^{(l)}\mathcal{P}^{(l)}$.

Why the rotation plane matters

The versor decomposition tells us that inter-layer dynamics are **metric-dominated**: the symmetric factor $\mathcal{P}^{(l)}$ contributes ∼94–97% of the transition operator's Frobenius norm, while the rotation factor $\mathcal{R}^{(l)}$ contributes only ∼3–6%.

This has a profound consequence for intervention design. A steering perturbation $\delta^{(l)}$ applied at layer l propagates to layer $l+1$ approximately as:

$$\delta^{(l+1)}_{\text{propagated}} \approx T^{(l)}\,\delta^{(l)} = \mathcal{R}^{(l)}\mathcal{P}^{(l)}\,\delta^{(l)}$$

Because $\mathcal{P}^{(l)}$ dominates, the propagated perturbation is shaped primarily by the *stretching directions* of $\mathcal{P}^{(l)}$, not by the rotation $\mathcal{R}^{(l)}$. A purely angular intervention at one layer acquires a radial component at the next:

$$\mathcal{P}^{(l)}\,\delta^{(l)}_{\text{angular}} = \underbrace{(\mathcal{P}^{(l)}\delta_{\text{angular}})_{\text{angular}}}_{\text{survives}} + \underbrace{(\mathcal{P}^{(l)}\delta_{\text{angular}})_{\text{radial}}}_{\text{generated by stretching}}$$

Common Pitfall

Per-layer isometry does not guarantee end-to-end isometry. A purely rotational intervention (e.g., Cayley steering) is exactly isometric at each layer. But the model's metric-dominated dynamics convert angular perturbations into radial ones. Over 28 layers, purely angular purity degrades substantially. Dense addition, which appears geometrically crude, can achieve higher end-to-end angular purity because $|v^\top \hat{H}| \sim \mathcal{O}(1/\sqrt{k})$ in high dimensions (because a random vector in k dimensions is nearly perpendicular to any fixed direction — the expected dot product shrinks as $1/\sqrt{k}$), making the perturbation naturally tangential.

The steering recipe

Key Idea

Plane-specific steering. Instead of modifying all directions at a target layer, intervene only in the **principal rotation plane**:

1. Identify target layers by effective work $D_l \cdot \theta^{(l)}$ (Chapter 12).
2. Extract the dominant bivector plane $\hat{B}^{(l)}$ at each target layer. Let (u_1, u_2) be its spanning eigenvectors.
3. Project $\mathcal{P}^{(l)}$ into the 2D subspace: $\mathcal{P}^{(l)}_{\text{plane}} = U_2^\top \mathcal{P}^{(l)} U_2$ where $U_2 = [u_1, u_2]$.
4. Modify only the eigenvalues of $\mathcal{P}^{(l)}_{\text{plane}}$ (amplify or suppress), leaving all other directions unchanged.

This is a rank-2 intervention in a k-dimensional space — maximally targeted. The intervention respects the model's own geometry: it acts in the plane where the model already performs its rotation.

Listing 13.2: Identifying steering targets

```
# Effective work per layer
eff_work = result.dependency_profile * result.angles[:n]

# Top-2 steering targets
top_layers = np.argsort(eff_work)[-2:][::-1]

for l in top_layers:
```

```
    vd = rf.decompositions[l]
    plane = vd.bivector.principal_planes(n_planes=1)[0]
    print(f"Layer {l}: D={result.dependency_profile[l]:.4f}, "
          f"theta={result.angles[l]:.4f}, "
          f"plane_angle={plane['angle']:.4f}")
    # plane['eigvecs'] gives the 2D subspace for intervention
```

Experiment: identifying steering targets

Running the steering recipe on Qwen2.5-7B with the factual prompt:

Layer	D_l	$\theta^{(l)}$	$D_l \times \theta^{(l)}$	Top plane angle
0	1.786	8.831	15.769	3.131
1	0.106	8.845	0.935	3.135
2	0.090	7.948	0.717	3.131

Layer 0 dominates overwhelmingly: its effective work (15.8) is 17× larger than layer 1's (0.94). This layer is the primary steering target.

Plane independence. The bivector similarity between the top-3 layers is near zero:

Layer pair	Bivector similarity
Layers 0 vs 1	+0.005
Layers 0 vs 2	+0.027
Layers 1 vs 2	−0.026

Interpretation. The near-zero similarities confirm that each layer rotates in an essentially *independent* plane. Steering at layer 0 will not interfere with the rotation at layers 1 or 2. At each target layer, the top-3 principal planes each carry ∼25% of the rotation weight, indicating compound (multi-plane) rotations. A plane-specific intervention at layer 0 would target the dominant 2D subspace, modifying only 2 of k dimensions — a maximally surgical intervention.

Comparison of steering methods

The following table summarises the geometric character of common steering approaches:

Method	Steering type	GA interpretation
Dense addition	$\delta = \alpha\, v$ (constant)	Naturally angular in high-k
Cayley/Rodrigues	$\delta = A\hat{H}$ (rotation)	Exactly grade-2 per layer
Sparse activation	$\delta = \lambda\, \Delta z$	Acts on individual interpretable directions (features identified via sparse autoencoders or similar methods)
Plane-specific	Modify $\mathcal{P}^{(l)}$ in 2D	Targeted grade-0 in one plane

What the Data Shows

Rotation as a mechanism of semantic steering (interpretive hypothesis, supported by initial experiments). The examples in this chapter suggest that context influence in language models is a rotational phenomenon. The Cayley bivector provides a unified representation: its magnitude detects vibing, its direction diagnoses the semantic category of context influence, and its associated rotation enables active intervention. In prior ablation studies on Qwen2.5-7B, random rotations catastrophically fail (0% context-following), while learned rotations succeed (83%), and scaling alone has no effect — consistent with the antisymmetric (bivector) component carrying the semantically meaningful signal. Establishing the generality of this finding across models and domains remains important future work.

Try It Yourself

Notebook: Chapter 13.

1. Run the context-ignoring detection on a factual question with and without a relevant paragraph. Does the curvature plane shift?
2. Generate the GA summary plot for a prompt you suspect will produce a hallucination. Look for the "busy but useless" layers.
3. Identify the top-2 steering targets (layer + plane) for a given prompt.

Sample output — the GA summary plot brings together all four diagnostic dimensions in a single view:

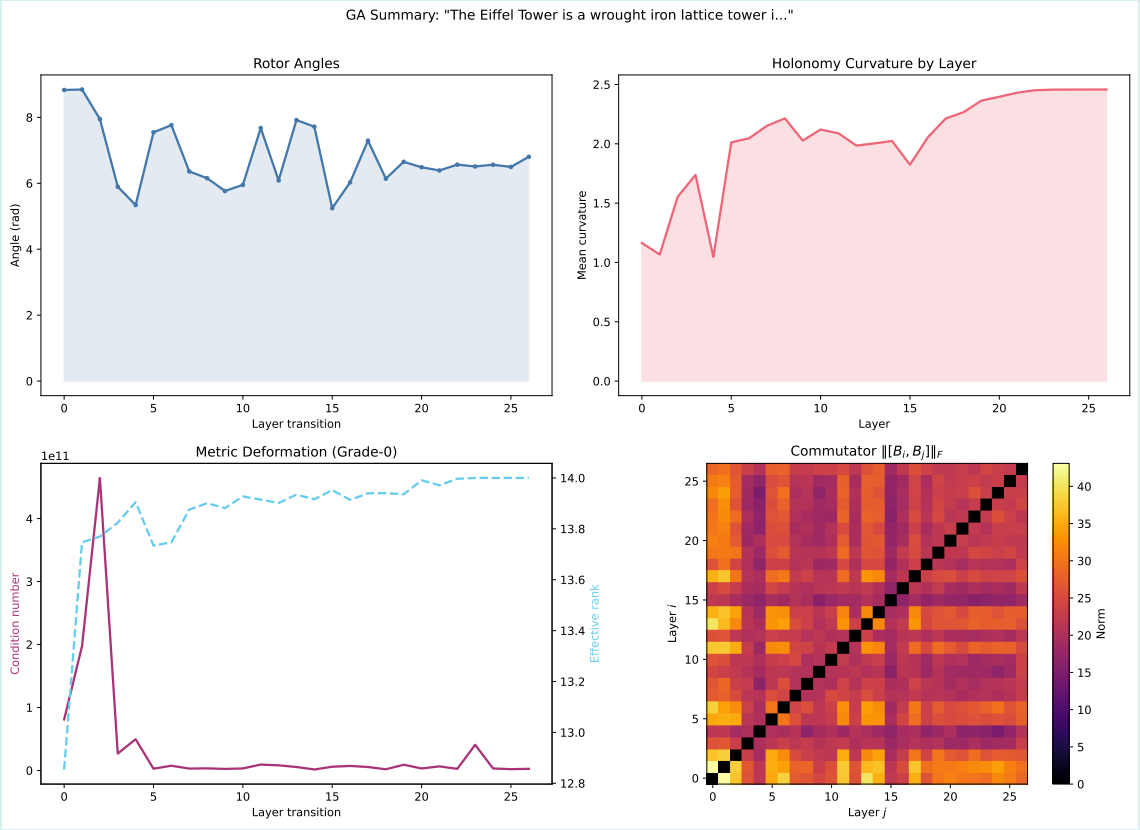


Figure 13.3: 4-panel GA summary for Qwen2.5-7B on a 14-token prompt. Top-left: rotor angles. Top-right: plane similarity. Bottom-left: curvature map. Bottom-right: capacity contributions.

CHAPTER 14

What Geometric Algebra Gave Us

14.1 The journey

You started with vectors and the dot product. You now have:

Chapter	GA concept	Transformer insight
1–2	Vectors, geometric product	Hidden states encode both alignment and plane information
3	Bivectors	Layer rotations have principal planes, not just magnitudes
4	Orthonormal frames	Whitening establishes the metric for valid GA
5	Rotors	Layer transitions are rotations with explicit planes
6	Versor decomposition	Grade-0 (stretch) vs. grade-2 (rotation)
7	Principal planes	Rotation plane evolution across the network
8	Eigenvalues	Grade-0 dominates control; late layers are highly selective
9	Commutators	Non-commutativity lives in specific planes
10	Holonomy	Curvature has a direction, not just a magnitude
11	Capacity	Decompose computational complexity by plane
12	Dependency + bivectors	Identify the planes that matter
13	Applications	Diagnose and steer via plane-specific interventions

14.2 What was invisible before

Scalar summaries — the size of a rotation ($\|U - I\|_F$), the size of a commutator ($\|[A_i, A_j]\|_F$), or the curvature magnitude ($\|\Omega - I\|_F$) — tell you *how much*. GA tells you *how much* and *in which direction*. Specifically:

1. Which planes each layer rotates in.
2. Which planes the curvature acts in.
3. Which planes different layers fail to commute in.

4. Which planes the model's output depends on.

None of this information is accessible from norms alone. It requires the full bivector.

14.3 Where to go next

1. **Higher grades.** We used only grades 0 and 2. Do trivectors (grade 3) carry meaningful information about three-layer interactions?

2. **Conformal GA.** Conformal GA extends the algebra by adding two extra dimensions (written $\mathrm{Cl}(k+1,1)$), which allows translations — not just rotations — to be represented algebraically. Could this unify residual connections (translations) with rotations (rotors)?

3. **GA in training dynamics.** How does the bivector field evolve during pre-training? Do specific planes emerge at specific training stages?

4. **Multimodal models.** When vision and language tokens interact, do they occupy different rotation planes?

5. **Projective GA.** Projective GA (written $\mathrm{Cl}(k,0,1)$) adds one extra dimension whose square is zero, providing a natural way to represent the affine shifts that residual connections add.

14.4 Further reading

- Doran and Lasenby, *Geometric Algebra for Physicists* (Cambridge, 2003). The most accessible GA introduction.
- Dorst, Fontijne, and Mann, *Geometric Algebra for Computer Science* (Morgan Kaufmann, 2007). Practical and code-oriented.
- Lounesto, *Clifford Algebras and Spinors* (Cambridge, 2001). The mathematical reference.
- Sudjianto and Zhang, *Layer–Time Geometry: How Language Models Process Information* (2026). The first-edition linear algebra treatment.
- Fang, Li, and Sudjianto, *Design and Modeling for Computer Experiments* (Chapman & Hall/CRC, 2006). Background on DOE methodology.

APPENDIX A

Transformers: A Primer

This appendix provides the transformer background needed for the book. No prior ML knowledge is assumed; we build from first principles.

Every model we study in this book — Qwen2.5-7B, Qwen2.5-32B, DeepSeek-R1-Distill-Qwen-7B — is a **GPT-style** (decoder-only, autoregressive) transformer. This is the architecture behind GPT-4, Claude, LLaMA, and most modern large language models. We describe this specific architecture below; encoder-only models (like BERT) and encoder-decoder models (like the original Transformer of Vaswani et al., 2017) share many ideas but differ in structure.

A.1 What is a language model?

A language model is a function that, given a sequence of words (tokens), predicts the next word. "The capital of France is" $\rightarrow$ "Paris." Internally, the model converts each token into a vector, transforms those vectors through many layers of computation, and reads out the prediction from the final layer. The key objects for us are the **hidden states** — the intermediate vectors at each layer.

A.2 Tokens and embeddings

Before processing, text is split into **tokens** (roughly word-pieces). Each token is assigned a vector $H^{(0,t)} \in \mathbb{R}^p$ from a learned **embedding table**. The integer t indexes the token position; p is the **hidden dimension** (e.g., $p = 3584$ for Qwen2.5-7B). These initial vectors are the starting point for all subsequent computation.

A.3 The decoder-only architecture

A GPT-style transformer consists of L identical **decoder layers** stacked sequentially (there is no separate encoder). Each layer transforms the hidden state $H^{(l,t)} \rightarrow H^{(l+1,t)}$ through two sub-components applied in sequence: *causal* self-attention and a feed-forward network.

Self-attention

Self-attention is the mechanism by which tokens communicate. At each layer, every token "looks at" all other tokens and computes a weighted combination of their representations. Here is the math.

Each hidden state $H^{(l,t)} \in \mathbb{R}^p$ is projected through three learned matrices to produce a **query**, **key**, and **value**:

$$q_t = W_Q\, H^{(l,t)}, \qquad k_t = W_K\, H^{(l,t)}, \qquad v_t = W_V\, H^{(l,t)}$$

where $W_Q, W_K \in \mathbb{R}^{d_k \times p}$ and $W_V \in \mathbb{R}^{d_v \times p}$ are learned weight matrices, and d_k, d_v are the query/key and value dimensions (typically $d_k = d_v = p/n_{\text{heads}}$).

Think of it this way: the query q_t asks "what am I looking for?", the key k_s says "here is what I contain," and the value v_s says "here is the information I provide if selected."

The **attention weight** from token t to token s measures how well the query matches the key:

$$\alpha_{t,s} = \frac{\exp(q_t^\top k_s \,/\, \sqrt{d_k})}{\sum_{r \le t} \exp(q_t^\top k_r \,/\, \sqrt{d_k})}$$

The division by $\sqrt{d_k}$ prevents the dot products from growing too large in high dimensions. The exp and normalisation (a **softmax**) turn the raw scores into probabilities that sum to 1. The constraint $r \le t$ is the **causal mask**: token t can only attend to tokens at or before its own position (not to future tokens).

The attention output for token t is the weighted sum of values:

$$\text{Attn}(t) = \sum_{s \le t} \alpha_{t,s}\, v_s$$

In practice, transformers use **multi-head attention**: the projection is split into n_{heads} independent "heads," each with its own W_Q, W_K, W_V matrices of smaller dimension. Each head can attend to different aspects of the input (one head might track syntactic relationships while another tracks semantic similarity). The outputs of all heads are concatenated and projected back to $\mathbb{R}^p$:

$$\text{MultiHead}(t) = W_O \begin{bmatrix} \text{head}_1(t) \\ \vdots \\ \text{head}_{n_h}(t) \end{bmatrix}$$

where $W_O \in \mathbb{R}^{p \times p}$ is a learned output projection.

The result: each token's representation is updated based on all tokens that precede it. The same word is represented differently depending on its context — "bank" in "river bank" vs. "bank account" will have different hidden states after attention.

Feed-forward network (MLP)

After attention, each token is processed *independently* by a two-layer neural network:

$$\text{FFN}(x) = W_2\, \sigma(W_1\, x + b_1) + b_2$$

where $W_1 \in \mathbb{R}^{d_{\text{ff}} \times p}$ and $W_2 \in \mathbb{R}^{p \times d_{\text{ff}}}$ are learned weight matrices, b_1, b_2 are biases, and σ is a nonlinear activation function (modern transformers typically use SiLU or GELU). The intermediate dimension d_{ff} is usually $4p$ or larger ($d_{\text{ff}} = 18944$ for Qwen2.5-7B).

The MLP is the model's "thinking" step for each token — it combines and transforms features without inter-token communication.

Residual connections and layer normalisation

Both sub-components use **residual connections**: the output is added to the input rather than replacing it:

$$H^{(l+1,t)} = H^{(l,t)} + \text{Attn}^{(l)}(t) + \text{FFN}^{(l)}(t)$$

This means the hidden state *accumulates* changes layer by layer rather than being overwritten. The residual structure is what makes the hidden-state grid continuous enough for GA to work: adjacent layers differ by a small additive update.

Layer normalisation (applied before each sub-component) rescales the hidden state to have zero mean and unit variance across its dimensions, stabilising training.

The hidden state $H^{(l,t)} \in \mathbb{R}^p$ at layer l and token position t is the model's internal representation — the central object of this book.

A.4 The hidden-state grid

The full set of hidden states forms a two-dimensional grid:

- **Layer axis**: $l = 0, 1, \ldots, L-1$, from input embedding to final representation.
- **Token axis**: $t = 0, 1, \ldots, T-1$, from the first token to the last.

Each grid point holds a p-dimensional vector. This $(L \times T)$ grid is the "laboratory" we study with GA. Moving along the layer axis corresponds to the model's processing depth; moving along the token axis corresponds to the model's processing breadth.

A.5 Autoregressive prediction

The defining feature of a decoder-only model is **autoregressive generation**: it predicts one token at a time, always conditioning on everything that came before:

$$P(x_t \mid x_1, x_2, \ldots, x_{t-1})$$

This is why the causal mask exists in self-attention — token t must not see tokens $t+1, t+2, \ldots$ because they have not been generated yet.

Concretely, the final hidden state $H^{(L-1,t)}$ at the last token position is projected to a score for every word in the vocabulary via a learned linear map $W_{\text{head}} \in \mathbb{R}^{V \times p}$:

$$\text{logits} = W_{\text{head}}\, H^{(L-1,t)}$$

where V is the vocabulary size (e.g., $V = 151{,}936$ for Qwen2.5-7B). These scores are called **logits**; higher values mean higher probability. Applying a softmax converts logits to a probability distribution over the vocabulary, and the highest-probability token is selected as the prediction.

The **score** $s(x, u)$ of a token u at the end of text x is the logit assigned to u — this is what the dependency gradient (Chapter 12) differentiates through.

To generate a full sentence, this process repeats: the predicted token is appended to the input, and the model runs again to predict the next token, one step at a time.

A.6 What gradients measure

A **gradient** is the derivative of a scalar output with respect to an input. If the model's score for token "Paris" is s, then $\partial s / \partial H^{(l,t)}$ is a vector pointing in the direction of hidden-state change that most increases the score for "Paris" at layer l, token t. Its magnitude tells you how *sensitive* the output is to changes at that point.

In this book, we use gradients to compute **dependency density** (Chapter 12): $D_{l,t} = \|\partial s / \partial H^{(l,t)}\|$. This tells us which layers and tokens matter most for the model's prediction — without altering the model or its output.

A.7 The models we study

- **Qwen2.5-7B**: 28 layers, hidden dimension 3584, 7 billion parameters. Our primary laboratory throughout the book.
- **Qwen2.5-32B**: 64 layers, hidden dimension 5120, 32 billion parameters. Used for scaling comparisons.
- **DeepSeek-R1-Distill-Qwen-7B**: same architecture as Qwen-7B, distilled from a larger reasoning model. Used to study how training objective affects geometric structure.

All models are freely available on Hugging Face and can be loaded with a single line of code (see the Preface). A GPU with 16 GB of memory is sufficient for the 7B models.

APPENDIX B

GA from Scratch

This appendix builds up Geometric Algebra from first principles. If you have taken a linear algebra course and are comfortable with vectors, dot products, and matrices, you have everything you need. We introduce one idea at a time, with small examples you can verify by hand.

B.1 The problem GA solves

Linear algebra gives you two operations on vectors: the **dot product** $a \cdot b$ (a scalar) and the **cross product** $a \times b$ (a vector, but only in 3D). The dot product tells you about alignment; the cross product tells you about the plane the two vectors span.

But the cross product is a 3D accident. In $\mathbb{R}^{256}$ (the kind of space we encounter inside transformers), there is no cross product. Yet we still want to talk about the plane that two vectors define, the rotation that maps one vector to another, and how rotations compose.

Geometric Algebra solves this by introducing a single product — the **geometric product** — that captures both alignment and plane information in any dimension. Everything else follows.

B.2 The geometric product: one rule

Start with a vector space $V = \mathbb{R}^k$ equipped with the standard inner product. The **geometric product** is an associative product on vectors defined by one rule:

GA Concept

The fundamental identity. For any vector v:

$$v\,v = v \cdot v = \|v\|^2$$

The geometric product of a vector with itself is its squared length.

That's it. This single rule, combined with associativity and distributivity, generates the entire algebra.

Let's see what this implies. Pick an orthonormal basis $e_1, e_2, \ldots, e_k$ (so $e_i \cdot e_j = \delta_{ij}$ (that is, $\delta_{ij} = 1$ when $i = j$ and 0 otherwise)). From the fundamental identity:

$$e_i\, e_i = e_i \cdot e_i = 1$$

Now consider the product $e_1\, e_2$. We can derive its properties. Compute $(e_1 + e_2)^2$ in two ways:

By the fundamental identity: $(e_1 + e_2)^2 = \|e_1 + e_2\|^2 = 2$

By expanding: $(e_1 + e_2)^2 = e_1e_1 + e_1e_2 + e_2e_1 + e_2e_2 = 1 + e_1e_2 + e_2e_1 + 1 = 2 + e_1e_2 + e_2e_1$

Comparing: $e_1e_2 + e_2e_1 = 0$, so

GA Concept

Anti-commutativity of orthogonal vectors.

$$e_i\, e_j = -e_j\, e_i \qquad (i \neq j)$$

Orthogonal vectors anti-commute under the geometric product.

For general vectors a and b, the geometric product decomposes as:

$$a\, b = \underbrace{a \cdot b}_{\text{scalar (grade-0)}} + \underbrace{a \wedge b}_{\text{bivector (grade-2)}}$$

The **inner product** $a \cdot b$ is the familiar dot product. The **outer product** $a \wedge b$ is something new: a *bivector*, which represents the oriented plane spanned by a and b.

A small example. In $\mathbb{R}^3$, let $a = 3e_1 + e_2$ and $b = e_1 + 2e_2$. Then:

$$\begin{aligned} a \cdot b &= 3 \cdot 1 + 1 \cdot 2 = 5 \\ a \wedge b &= (3 \cdot 2 - 1 \cdot 1)\, e_1 \wedge e_2 = 5\, e_1 \wedge e_2 \end{aligned}$$

So $a\, b = 5 + 5\, e_1 \wedge e_2$. The scalar part says the vectors are partially aligned. The bivector part says they span an area of 5 in the e_1e_2 plane.

B.3 The Clifford algebra $\mathrm{Cl}(k, 0)$

The algebra generated by all products of basis vectors $e_1, \ldots, e_k$ under the geometric product is the **Clifford algebra** $\mathrm{Cl}(k, 0)$. The "$(k, 0)$" means k dimensions where every basis vector squares to $+1$ (as opposed to signatures used in relativity, which we never need). Here, 'metric' just means the rule for computing squared lengths — the standard dot product.

The distinct basis elements are:

Grade	Basis elements	Count	Name
0	1	$\binom{k}{0} = 1$	Scalars
1	$e_1, e_2, \ldots, e_k$	$\binom{k}{1} = k$	Vectors
2	$e_1e_2,\ e_1e_3,\ \ldots$	$\binom{k}{2} = \frac{k(k-1)}{2}$	Bivectors
3	$e_1e_2e_3,\ \ldots$	$\binom{k}{3}$	Trivectors
$\vdots$	$\vdots$	$\vdots$	$\vdots$
k	$e_1e_2\cdots e_k$	1	Pseudoscalar

The total dimension is $\sum_{r=0}^{k}\binom{k}{r} = 2^k$. A general element of $\mathrm{Cl}(k,0)$ is a **multivector**: a sum of components across different grades.

For $k = 3$: $\mathrm{Cl}(3,0)$ has dimension $2^3 = 8$, with 1 scalar, 3 vectors, 3 bivectors, and 1 pseudoscalar.

For $k = 256$ (the whitened dimension used in this book): $\mathrm{Cl}(256,0)$ has dimension 2^{256}, an astronomically large space. But we only ever use grades 0, 1, and 2 — scalars, vectors, and bivectors — which is a subspace of dimension $1 + 256 + 32{,}640 = 32{,}897$.

B.4 Blades and multivectors

A **blade** is a grade-r element that can be written as a single outer product:

$$B_r = v_1 \wedge v_2 \wedge \cdots \wedge v_r$$

A blade represents an oriented r-dimensional subspace. For grade-2 (bivectors), a blade is a single oriented plane. A general bivector may be a sum of blades — this happens when a rotation involves multiple planes simultaneously.

Example: simple vs compound bivectors.

- $B = 3\,e_1 \wedge e_2$ is a **simple** bivector (a blade): it represents one plane.
- $B = 2\,e_1 \wedge e_2 + 5\,e_3 \wedge e_4$ is a **compound** bivector: it involves two independent planes. This cannot be written as a single $v_1 \wedge v_2$.

In 3D, every bivector is simple (there is only one independent plane). In 4D and above, compound bivectors exist and are common. In the 256-dimensional space of transformer hidden states, layer transition bivectors are highly compound — they involve dozens of active planes. This is where the "principal plane decomposition" from Chapter 3 becomes essential.

B.5 Rotors: the GA way to rotate

A **rotor** is the GA object that represents a rotation. It is built from scalars and bivectors only (no plain vectors — this is called 'even-grade'). It must satisfy $R\widetilde{R} = 1$, where $\widetilde{R}$ reverses the order of

all vector factors (defined precisely in the Reversion section below). Think of a rotor as the GA analog of a unit-length complex number $e^{i\theta}$.

The simplest rotor rotates by angle θ in the plane of unit bivector $\hat{B}$:

$$R = \exp\left(-\frac{\hat{B}\theta}{2}\right) = \cos\left(\frac{\theta}{2}\right) - \hat{B}\,\sin\left(\frac{\theta}{2}\right)$$

Why the half angle? Because a rotor acts on a vector v via the **sandwich product**:

$$v' = R\,v\,\widetilde{R}$$

The two factors of R each contribute $\theta/2$, giving a total rotation of θ.

Example in $\mathbb{R}^3$. To rotate e_1 by $90°$ in the e_1e_2 plane:

$$\begin{aligned} R &= \cos(45°) - e_1e_2\sin(45°) = \frac{\sqrt{2}}{2}(1 - e_1e_2) \\ R\,e_1\,\widetilde{R} &= \frac{1}{2}(1 - e_1e_2)\,e_1\,(1 + e_1e_2) \\ &= \frac{1}{2}(e_1 - e_1e_2e_1)(1 + e_1e_2) \\ &= \frac{1}{2}(e_1 + e_2)(1 + e_1e_2) \\ &= e_2 \quad \checkmark \end{aligned}$$

(We used $e_1e_2e_1 = -e_2e_1e_1 = -e_2$ and similar rules.)

Multi-plane rotors. In high dimensions, a rotor can involve multiple independent rotation planes simultaneously. A general rotor in $\mathbb{R}^k$ is:

$$R = \exp\left(-\frac{1}{2}\sum_j \theta_j\hat{B}_j\right)$$

where each $\hat{B}_j$ is a unit bivector in a distinct plane. If the planes are orthogonal (commuting bivectors), this factorises into a product of simple rotors: $R = R_1R_2\cdots R_m$.

B.6 The spin group

The set of all rotors forms a group under the geometric product, called the **spin group** $\mathrm{Spin}(k)$.

Key properties:

- **Closure**: the product of two rotors is a rotor.
- **Identity**: $R = 1$ (the scalar 1 is a rotor with zero rotation).

- **Inverse**: $R^{-1} = \widetilde{R}$ (just reverse the rotor).
- SO(k): In matrix language, SO(k) is the group of $k \times k$ orthogonal matrices with determinant $+1$ — in other words, the group of all rotations in k dimensions.
- **Double cover**: both R and $-R$ produce the same rotation (since $(-R)v(-\widetilde{R}) = Rv\widetilde{R}$). This means there are always two rotors for every rotation matrix — a bookkeeping detail that never affects computations. So Spin(k) is a double cover of SO(k).

The spin group provides a more compact and algebraically natural parameterisation: instead of k^2 matrix entries subject to $k(k+1)/2$ orthogonality constraints, a rotor is determined by a bivector (the logarithm), which has exactly $k(k-1)/2$ free parameters — the right number for a rotation in k dimensions.

B.7 Bivectors as rotation generators

(the Lie algebra $\mathfrak{so}(k)$)

Bivectors do double duty: they represent oriented planes *and* they are the building blocks of rotations: a bivector B describes the direction and rate of a rotation, and exponentiating gives the full rotation $R = e^{-B/2}$. The bivector is to the rotation what velocity is to position.

Given two bivectors (two rotation directions), we can ask: does the order matter? The **commutator** answers this question. The set of all bivectors, equipped with the **commutator bracket**

$$[B_1, B_2] = B_1 B_2 - B_2 B_1$$

forms an algebraic structure called a **Lie algebra** — a vector space with a bracket operation that measures non-commutativity. This particular Lie algebra is the one that governs rotations in k dimensions, known as $\mathfrak{so}(k)$.

The connection is explicit: a bivector $B = \sum_{i<j} B_{ij}\, e_i \wedge e_j$ corresponds to the skew-symmetric matrix with entries $A_{ij} = B_{ij}$, $A_{ji} = -B_{ij}$. The commutator of bivectors maps directly to the matrix commutator $[A_1, A_2] = A_1 A_2 - A_2 A_1$.

This Lie algebra has dimension $k(k-1)/2$. For $k = 256$, that is 32,640 — the number of independent bivector components, and equivalently the number of independent rotation planes.

The commutator $[B_1, B_2]$ measures how much two rotations fail to commute. If $[B_1, B_2] = 0$, the rotations are independent (they can be applied in either order). If $[B_1, B_2] \neq 0$, the order matters — and the commutator bivector tells you *which plane* the non-commutativity lives in.

B.8 Reversion

The **reverse** $\widetilde{M}$ of a multivector reverses the order of all vector factors in each term:

$$\widetilde{(v_1 v_2 \cdots v_r)} = v_r \cdots v_2 v_1$$

For a grade-r element, this introduces a sign: $\widetilde{B_r} = (-1)^{r(r-1)/2} B_r$. In practice:

- Grade 0 (scalar): $\widetilde{s} = s$
- Grade 1 (vector): $\widetilde{v} = v$
- Grade 2 (bivector): $\widetilde{B} = -B$
- Grade 3 (trivector): $\widetilde{T} = -T$

Reversion is the GA analog of matrix transposition. For a rotor $R = \cos(\theta/2) - B\sin(\theta/2)$, the reverse is $\widetilde{R} = \cos(\theta/2) + B\sin(\theta/2) = R^{-1}$, which is the inverse rotation.

B.9 Duality and the pseudoscalar

The **pseudoscalar** is the highest-grade basis element:

$$I_k = e_1\, e_2\, \cdots\, e_k$$

It has a remarkable property: multiplying by I_k maps a grade-r element to a grade-$(k-r)$ element. This is called **duality**.

In 3D. $I_3 = e_1 e_2 e_3$ has $I_3^2 = -1$. The dual of a bivector is a vector:

$$(e_1 \wedge e_2)\, I_3^{-1} = e_3$$

This is exactly the cross product: $a \times b = (a \wedge b)\, I_3^{-1}$. The cross product is secretly a bivector in disguise, projected down to a vector via duality.

In higher dimensions. There is no cross product in $\mathbb{R}^k$ for $k > 3$ (the dual of a bivector is a $(k-2)$-vector, not a vector). But the bivector itself is perfectly well-defined. This is the key advantage of GA: bivectors work in any dimension, while the cross product is stuck in 3D.

B.10 What we use in this book

This book restricts attention to three grades:

- **Grade 0**: scalars — inner products, stretching factors, eigenvalues.
- **Grade 1**: vectors — hidden states, basis vectors.
- **Grade 2**: bivectors — rotation planes, commutators, curvature, holonomy.

We never need trivectors or higher-grade elements, because the operations we study (rotations, stretching, parallel transport) live entirely in the sub-algebra generated by grades 0 and 2 (scalars and bivectors). In the literature this is called the **even sub-algebra** $\mathrm{Cl}^+(k, 0)$, but you do not need this name to follow the book.

The matrix representations used throughout the book are:

- Vectors: $\mathbb{R}^k$ arrays.
- Bivectors: $k \times k$ skew-symmetric matrices ($A_{ij} = -A_{ji}$).
- Rotors: $k \times k$ orthogonal matrices ($RR^\top = I$), with the bivector recovered via the matrix logarithm, Cayley map, or Rodrigues formula.

This is why the `layer_time_ga` library stores bivectors as NumPy arrays rather than as 2^k-dimensional multivector objects: in the grade-$\{0, 1, 2\}$ sector, the matrix representation is both compact and computationally efficient.

APPENDIX C

The Matrix–Multivector Bridge

Matrix operation	**GA equivalent**	**What GA adds**
Dot product $a^\top b$	Inner product $a \cdot b$	Same computation
$ab^\top - ba^\top$ (skew)	Outer product $a \wedge b$	Named as a bivector (oriented plane)
Skew-symmetric A	Bivector B	Principal plane decomposition
Orthogonal matrix U (rotation, $\det = +1$)	Rotor $R = e^{-B/2}$	Explicit plane + angle
Polar decomposition $T = UP$	Versor decomposition $T = RP$	Rotation (plane information) cleanly separated from stretching (magnitude information)
$\|U - I\|_F$	Rotor angle θ	Plus: which plane
$A_i A_j - A_j A_i$	$[B_i, B_j]$	Plus: decompose into planes
$\|\Omega - I\|_F$ (curvature)	Holonomy rotor	Plus: curvature direction
Eigenvalues of P	Grade-0 content	Natural grade hierarchy

APPENDIX D

Code Reference

D.1 Student API: `ltg_ga`

```python
import ltg_ga

model = ltg_ga.load_model("Qwen/Qwen2.5-7B")
result = ltg_ga.analyse("prompt", model=model)
result.summary()
result.plot_ga_summary(save_path="summary.png")

# Compare multiple prompts
ltg_ga.compare([r1, r2, r3], save_path="compare.png")

# Capacity analysis
cap = ltg_ga.capacity("prompt", model=model)
```

D.2 Core algebra: `layer_time_ga.algebra`

```python
from layer_time_ga.algebra import (
    Bivector, Rotor,
    bivector_from_skew, rotor_from_orthogonal,
    rotor_compose, rotor_inverse,
    commutator_bivector, grade_decomposition,
    geometric_product_vectors,
)
```

D.3 Decomposition: `layer_time_ga.decomposition`

```python
from layer_time_ga.decomposition import (
    VersorDecomposition, LayerRotorField,
```

```
    versor_decompose, extract_rotor_field,
    extract_bivector_field, extract_metric_field,
)
```

D.4 Curvature: `layer_time_ga.curvature`

```
from layer_time_ga.curvature import (
    holonomy_rotor, holonomy_field, holonomy_scalar_map,
    commutator_field, commutator_bivectors,
    commutator_plane_decomposition,
)
```

D.5 Capacity: `layer_time_ga.capacity`

```
from layer_time_ga.capacity import (
    ga_capacity_profile, GACapacityProfile,
)
```

Glossary

Term	Definition
Bivector	A grade-2 element representing an oriented plane; stored as a skew-symmetric matrix.
Blade	A bivector (or higher-grade element) that can be written as a single outer product $a \wedge b$; represents one plane.
Clifford algebra	The algebra $\mathrm{Cl}(k, 0)$ generated by vectors under $v^2 = \|v\|^2$ (i.e., squaring a vector gives its squared length). Contains all grades from 0 to k.
Commutator	$[B_1, B_2] = B_1 B_2 - B_2 B_1$; a bivector measuring non-commutativity.
Curvature bivector	$\log(R_{\text{loop}})$: the plane in which space is curved at a grid point; obtained from the holonomy rotor.
Geometric product	$ab = a \cdot b + a \wedge b$; the fundamental operation of GA.
Grade	The "type" of an algebraic element: 0 = scalar, 1 = vector, 2 = bivector.
Holonomy	The rotor obtained by composing rotors around one cell of the layer-token grid; measures curvature.
Metric factor	The symmetric positive-definite matrix P from $T = RP$; encodes grade-0 stretching.
Orthonormal frame	Basis vectors with $e_i \cdot e_j = 1$ when $i = j$ and 0 otherwise; required for valid GA operations.
Principal plane	A simple-bivector component of a compound bivector; extracted via eigendecomposition.
Reversion	The GA analog of transpose: $\widetilde{R} = R^{-1}$ for rotors.
Rotor	An even-grade element $R = \exp(-B\theta/2)$ representing a rotation in plane B by angle θ.
Sandwich product	$v' = Rv\widetilde{R}$; how a rotor acts on a vector.
Versor decomposition	$T = RP$: separating rotation (grade-2) from stretching (grade-0).
Whitening	PCA-based dimensionality reduction (PCA projection followed by unit-variance rescaling) that constructs an orthonormal frame.